**POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY**
**Faculty of Information Technology**

# Intelligent System Development

## GRAPH 01

### A Diabetes Knowledge Graph from Vinmec Health Articles with Neo4j

**Instructor / Lecturer:** Assoc. Prof. Dinh Que Tran, Ph.D.
**Email:** quetd@ptit.edu.vn, tdque@yahoo.com

**Student Name:** Lưu Anh Dũng
**Student ID:** B23DCDK036
**Semester:** I.2025 – 2026
**Submission:** Individual Project

**Hanoi, 2026**

# Contents


**Summary** 2

**1 Introduction** 3
- 1.1 The Application Problem . . . 3
- 1.2 Why Conventional Keyword Search Is Insufficient . . . 3
- 1.3 Why a Knowledge Graph . . . 3
- 1.4 Objectives and Scope . . . 4
- 1.5 Summary of the Approach . . . 4

**2 Problem Definition** 5
- 2.1 Input and Output . . . 5
- 2.2 Questions the System Must Answer . . . 5
- 2.3 Success Criteria . . . 6
- 2.4 What the System Is Not . . . 6

**3 Data Collection** 6
- 3.1 Why Vinmec . . . 6
- 3.2 Finding the Pages . . . 7
- 3.3 Why a Second Filter Is Needed . . . 7
- 3.4 Parsing a Page . . . 8
- 3.5 Data Volume . . . 9

**4 Knowledge Graph Design** 10
- 4.1 Facts, Entities and Relations . . . 10
- 4.2 Schema: Entity Types and Relation Types . . . 11
- 4.3 Extraction Method . . . 13
  - 4.3.1 Why not let each bullet point become a node . . . 13
  - 4.3.2 The vocabulary and entity normalisation . . . 14
  - 4.3.3 Finding an alias in the text . . . 14
  - 4.3.4 The section rule: the heading decides what may be extracted . . . 15
  - 4.3.5 Which diabetes is the subject? . . . 16
  - 4.3.6 Counting evidence: why one page is not enough . . . 17
  - 4.3.7 Foods: recommended or to avoid . . . 18
  - 4.3.8 Drug side effects . . . 18
  - 4.3.9 Background facts added by hand . . . 18
- 4.4 Output Files . . . 19
- 4.5 Meeting the Lecture's Requirements . . . 19

**5 System Implementation** 20
- 5.1 Project Structure . . . 20
- 5.2 Storing the Graph in Neo4j . . . 20
  - 5.2.1 Why a graph database . . . 20
  - 5.2.2 Data model . . . 20
  - 5.2.3 Loading . . . 21
- 5.3 Querying with Cypher . . . 21
- 5.4 Graph Retrieval . . . 22
- 5.5 The Website . . . 22
  - 5.5.1 Architecture . . . 22
  - 5.5.2 API . . . 23
  - 5.5.3 Web client . . . 24

**6 Visualisation** 24
- 6.1 Size of the Graph . . . 25
- 6.2 The Whole Graph . . . 26
- 6.3 Focused Views . . . 26
- 6.4 Comparing the Diabetes Types . . . 29
- 6.5 Which Entities Are the Most Important? . . . 30
- 6.6 What Vinmec Writes About Most . . . 31

**7 Experimental Evaluation** 31

7.1 Query Results . . . 31
7.1.1 One-hop: everything about Metformin . . . 31
7.1.2 Two-hop: which organs can type 2 diabetes damage? . . . 32
7.1.3 Comparing type 1 and type 2 symptoms . . . 32
7.1.4 A symptom checker . . . 33
7.1.5 Diagnosis: tests and thresholds . . . 33
7.1.6 Diet for gestational diabetes . . . 34
7.1.7 Provenance: where does a fact come from? . . . 35
7.1.8 Co-occurrence through articles . . . 35
7.2 Correctness of the Facts . . . 35
7.2.1 What was measured and how . . . 35
7.2.2 Result . . . 36
7.3 How Much Evidence the Facts Have . . . 37

**8 Demo** **37**
8.1 Scenario 1: Finding an Entity . . . 37
8.2 Scenario 2: Everything About One Entity, With Its Sources . . . 39
8.3 Scenario 3: Comparing the Types of Diabetes . . . 40
8.4 Scenario 4: Checking Symptoms . . . 41
8.5 Scenario 5: How Are Two Things Connected? . . . 42
8.6 On a Phone . . . 43

**9 Error Analysis** **43**
9.1 The Two Errors in the Sample . . . 44
9.2 The Fix . . . 44
9.3 What the Fix Cost: Precision Versus Recall . . . 44
9.4 Other Known Errors . . . 45

**10 Limitations** **46**

**11 Conclusion and Future Work** **46**
11.1 What Was Achieved . . . 46
11.2 Future Work . . . 47

**A Reproduction** **47**

## Summary

This project turns the diabetes articles published on the website of the Vinmec hospital group into a **Knowledge Graph**: a network of small, explicit facts such as *(Đái tháo đường type 2, HAS_SYMPTOM, Khát nhiều)* or *(Metformin, BELONGS_TO_CLASS, Biguanide)*. The facts are stored in the graph database **Neo4j**, can be queried with the Cypher language, and can be explored by anyone through a small **website** that shows every fact together with the articles it comes from.

The pipeline has four stages. A crawler reads Vinmec's sitemap, selects 1,197 candidate pages whose address mentions diabetes, downloads them politely and keeps the 1,096 that really discuss the disease (about 1.13 million words). An extractor reads each article section by section and uses the section heading (*Triệu chứng*, *Nguyên nhân*, *Biến chứng*, *Điều trị*…) to decide what kind of fact the text below it can contain; it then matches a hand-written vocabulary of 106 diabetes concepts written in 320 different ways. Each article votes for the facts it states, and a fact is kept only when at least two different articles agree. The result, loaded into Neo4j, has **127 entities of 10 types and 339 facts of 12 relation types**, linked to the 1,096 source articles.

Four results stand out:

1. **Section headings are the key signal.** Health articles are organised under standard headings, and using them turns free Vietnamese text into typed facts without any language model. A word found in the wrong section ("mệt mỏi" in a breakfast article) is simply ignored.

2. **Agreement between articles removes noise.** Requiring support from at least two articles drops one-off mentions, and the support count stored on every fact lets the user choose how cautious to be: 252 facts are stated by three or more articles, 70 by ten or more.

3. **The facts are reliable.** On a random sample of 50 extracted facts, each checked against all its source articles, **48 were correct (0.96; likely range 0.87–0.99)**. Both errors had the same cause—a fact about a complication attached to the disease itself—which led to a stricter heading rule and a clear analysis of the trade-off it brings.

4. **The graph answers questions no single article answers.** Following two arrows—type 2 diabetes *can cause* diabetic kidney disease, which *affects* the kidney—answers "which organs can type 2 damage?", and every answer can be traced back to the articles that state each step.

The project meets every requirement of the lecture's Mini Project 3 (at least 50 entities, 100 relationships, 5 relationship types, 3 entity types and 5 Cypher queries) and of Exercise 1 (triples stored as CSV, visualisation, one-hop and two-hop queries).

# 1 Introduction

## 1.1 The Application Problem

Diabetes is one of the most common chronic diseases in Vietnam, and people who want to understand it usually read health articles on the internet. Vinmec, one of the largest private hospital groups in the country, publishes almost a thousand Vietnamese articles about diabetes alone: what its symptoms are, who is at risk, which complications it causes, how it is diagnosed, which drugs treat it, what patients should eat.

This knowledge is valuable, but it is locked inside text. A person who wants to know *which complications of type 2 diabetes affect the eyes* has to find the right articles, read them, and put the pieces together themselves. The same symptom is written in different ways in different articles (*uống nhiều*, *khát nước*, *hay khát*), and the answer to a question is often spread across several articles: one article says that type 2 diabetes can cause retinopathy, another explains that retinopathy damages the retina of the eye. A computer program has an even harder time: for it, an article is only a long sequence of characters.

The goal of this project is to turn this collection of articles into a form that a program can query directly and that a person can explore without reading hundreds of pages, while keeping the link to the original articles so that every answer can be checked.

## 1.2 Why Conventional Keyword Search Is Insufficient

The obvious tool for finding information in a collection of documents is keyword search. It fails here for three reasons.

**The same thing is written in many ways.** A search for *khát nước* (thirsty) misses an article that writes *uống nhiều* (drinking a lot) or *khát nhiều*. A search for *Metformin* misses the brand name *Glucophage*. Every reader would have to know all the synonyms in advance.

**A word is not a fact.** An article about breakfast may say that eating well "helps you feel less *mệt mỏi*" (tired). A keyword search for *mệt mỏi* finds this article, but the article does not say that fatigue is a symptom of diabetes. Keyword search finds where a word appears, not what the text says about it.

**Answers that need two facts cannot be found.** No article contains the sentence "type 2 diabetes damages the kidney" in exactly that form; the answer comes from combining "type 2 can cause diabetic kidney disease" with "diabetic kidney disease affects the kidney". Keyword search returns documents one at a time and has no way to combine statements from different documents.

## 1.3 Why a Knowledge Graph

A Knowledge Graph stores knowledge not as sentences but as many small, separate **facts**, each connecting exactly two things with a named relationship: *(subject, relation, object)*. Drawn as a picture, the things are dots (**entities** or nodes) and the facts are arrows between them (**relationships** or edges), so the whole collection forms a network—a graph.

This representation answers the three problems above directly:

- **Synonyms are resolved once.** When the facts are created, *uống nhiều*, *khát nước* and *hay khát* are all mapped to one entity, *Khát nhiều*. Every later question uses that single entity.
- **Facts are explicit and typed.** The graph only contains a fact when an article states it in a place where such a statement is made (Section 4.3), so "fatigue is a symptom of diabetes" is a fact only if articles really say so.
- **Facts can be combined by following arrows.** A question becomes a walk through the network: start at *type 2*, follow *can cause* to the complications, then *affects* to the organs. The path itself is the explanation of the answer.

The lecture notes summarise the difference between the two kinds of retrieval: vector or keyword retrieval asks "which passages are similar to the question?", while graph retrieval asks "which entities and relationships are explicitly connected?". For questions about how symptoms, causes, complications, tests and treatments relate to each other, the second question is the right one.

A Knowledge Graph also keeps something a trained model loses: **provenance**. Every fact in this project stores the articles that state it, so any answer can be checked against its sources. For a medical topic, where a wrong answer can mislead a patient, this is essential.

### 1.4 Objectives and Scope

The task is to build a Knowledge Graph about diabetes from Vinmec data, store and query it with Neo4j, and provide a website to retrieve the data. Table 1 lists the requirements, taken from the task and from the lecture's Mini Project 3 and Exercise 1, and the section of this report that addresses each one.

Table 1: Requirements and where they are addressed.

| # | Requirement | Section |
|---|---|---|
| R1 | Collect data about diabetes from Vinmec | 3 |
| R2 | Build a Knowledge Graph: $\geq$ 50 entities, $\geq$ 100 relationships, $\geq$ 5 relationship types, $\geq$ 3 entity types | 4, 4.5 |
| R3 | Store the triples as CSV in the form `subject, relation, object` | 4.4 |
| R4 | Store and query the graph in Neo4j; at least 5 Cypher queries, including one-hop and two-hop | 5.2, 7.1 |
| R5 | Visualise the graph | 6 |
| R6 | Graph retrieval starting from an entity, limited to 1–2 hops | 5.4 |
| R7 | A website for users to retrieve the data | 5.5, 8 |
| R8 | Evaluate the system systematically | 7 |
| R9 | Analyse errors and limitations | 9, 10 |

The structure of the report follows the lecture's *Recommended Final Report Structure* (§37 of *From Deep Learning to Knowledge Graph, RAG and Intelligent Agents*). That outline is written for a complete intelligent assistant and also contains sections on a deep-learning model, a RAG architecture and an agent architecture. This project covers the Knowledge Graph part and a retrieval website, so those three sections are not included. Two sections are added instead because they carry a large part of the work: **Visualisation** (Section 6) and **Demo** (Section 8), which walks through the website with screenshots.

### 1.5 Summary of the Approach

Figure 1 shows the whole system. Each box is a separate program, and each one reads the output of the previous one from disk. The stages are separated on purpose: crawling takes about twenty minutes and depends on the website being reachable, so it is done once; extraction is where the rules were changed and re-tested several times, so it must be cheap to re-run without downloading anything; loading into Neo4j can rebuild the database from the CSV files at any time; and the website only reads from the database.

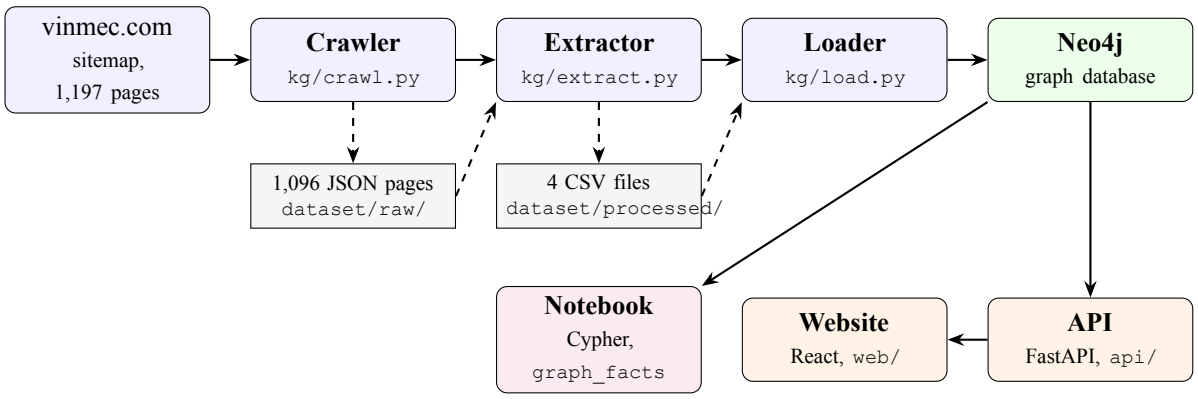



Figure 1: Overview of the system. Solid arrows: the flow of data between programs; dashed arrows: files written to and read from disk. The notebook, the Cypher queries and the website all read the same Neo4j database.

# 2 Problem Definition

## 2.1 Input and Output

**Input.** The public Vietnamese articles of vinmec.com that are about diabetes: Vinmec's official disease pages (short structured summaries with fixed headings: overview, causes, symptoms, risk groups, prevention, diagnosis, treatment) and the much larger set of news and advice articles written or reviewed by Vinmec doctors. No other source is used.

**Output.** A Knowledge Graph in which

- every node is a clearly named diabetes concept with a type: a type of diabetes, a symptom, a risk factor, a complication, a diagnostic test, a drug, a drug class, a food, a lifestyle measure or an organ;
- every edge is a fact with a named relation, such as `HAS_SYMPTOM` or `TREATED_BY`;
- every extracted fact records how many different articles state it (its **support**) and which articles they are;

plus three ways to use it: Cypher queries in Neo4j, a retrieval function that returns the facts around one entity, and a website.

## 2.2 Questions the System Must Answer

A Knowledge Graph is only useful if it answers real questions. The design started from the questions a reader of diabetes articles would ask, and each one corresponds to a different kind of graph query:

Table 2: Questions the system must answer and the kind of graph query each one needs.

| Question | Query type | What the graph must contain |
|---|---|---|
| What are the symptoms of type 1 diabetes? What drugs treat type 2? | one-hop | typed facts from each diabetes type |
| What do we know about Metformin? | one-hop, both directions | facts in which Metformin is the subject or the object |
| Which organs can type 2 diabetes damage? | two-hop | complications *and* the organ each complication affects |
| Which types of diabetes match these symptoms? | aggregation over one hop | symptom facts per type, with their support |
| How is obesity connected to the kidney? | shortest path | a connected graph |
| How is diabetes diagnosed, and at which values? | one-hop + node property | tests, with thresholds stored on the test nodes |
| Where does this fact come from? | provenance | article URLs stored on each fact |

These questions decided the design. The two-hop question is the reason for the hand-added *affects organ* facts (Section 4.3.9); the symptom checker is the reason why support is stored on every fact rather than used once as a filter; the provenance question is the reason why the articles themselves are nodes in the graph.

### 2.3 Success Criteria

1. **Size and structure** (lecture Mini Project 3 and Exercise 1): at least 50 entities, 100 relationships, 5 relationship types and 3 entity types; triples in CSV; at least five Cypher queries including one-hop and two-hop queries; a visualisation.
2. **Correctness**: measured, not assumed. A random sample of extracted facts is checked by hand against the source articles, and the share of correct facts (precision) is reported with its uncertainty.
3. **Traceability**: every extracted fact links to the articles that state it.
4. **Usability**: a person without any knowledge of Cypher can find an entity, see its facts and their sources, compare diabetes types, check symptoms and find connections, through a website.

### 2.4 What the System Is Not

The graph summarises what public health articles say. It is not a diagnostic tool and not medical advice. "Correct" in this report means that the Vinmec articles state the fact for that type of diabetes, not that a doctor has verified it against current clinical guidelines. This distinction is repeated in the website and in the evaluation, because for a medical topic it matters.

## 3 Data Collection

### 3.1 Why Vinmec

Three properties made Vinmec a good single source for a Vietnamese diabetes graph. First, the language: the articles are written in Vietnamese, so the graph uses the words Vietnamese readers actually use. Second, coverage: the site has almost a thousand diabetes articles covering every aspect of the disease, from early signs to diet, drugs and pregnancy. Third, consistency: the articles are written or reviewed by the hospital's doctors and follow a recognisable structure, with headings such as *Triệu chứng* (symptoms), *Nguyên nhân* (causes) and *Điều trị* (treatment). Section 4.3 shows that this structure is what makes automatic extraction possible.

## 3.2 Finding the Pages

**Why not the search box.** The obvious way to find diabetes articles would be the website's search box. That does not work for a program. Vinmec's search results are produced by JavaScript running inside the browser after the page has loaded. When a program downloads the search page directly, it receives the page *before* the JavaScript has run, and there are no results in it. A test request for "tiểu đường" returned a 150 KB page containing no article links.

**The sitemap.** Large websites publish a **sitemap**, a file listing every URL on the site, so that search engines can find all their pages. Vinmec's `sitemap.xml` points to about 230 smaller sitemaps. The crawler reads the ones for Vietnamese articles (`posts-vi-1.xml` … `posts-vi-87.xml`) and Vietnamese disease pages (`diseases-vi-1.xml`), which together list about 222,000 URLs—the entire Vietnamese website.

**Selecting by slug.** Every URL ends with a short readable name of the page, its **slug**, written without Vietnamese accents. The article *"Bệnh đái tháo đường: chế độ ăn như thế nào?"* has the slug `benh-dai-thao-duong-che-do-nhu-nao-vi`. This makes it possible to select diabetes pages without downloading all 222,000. Keywords are split into two strengths because they carry different risks:

- **strong keywords** (`tieu-duong`, `thao-duong`, `dai-duong`): a slug containing one of them is almost certainly about diabetes;
- **weak keywords** (`insulin`, `metformin`, `hba1c`, `duong-huyet` = blood sugar, `glucose`, `sglt2`, …): the page *might* be about diabetes, but these words also appear in articles about other topics, such as the role of insulin in the body or blood sugar in healthy people.

The slug `thao-nhat` is excluded explicitly: *đái tháo nhạt* (diabetes insipidus) shares the words "đái tháo" with *đái tháo đường* but is a different disease of the kidneys and hormones, unrelated to blood sugar.

This selection gives **1,197 candidate URLs**. Six of them either failed to download or were the same article published under a second address (the crawler skips a page whose title it has already seen), so **1,191 pages** were downloaded and checked: 944 with a strong keyword and 247 with only a weak one.

**Politeness.** The site's `robots.txt` (where a website states what automated programs may access) allows these paths. The crawler identifies itself as a student research crawler, waits one second between requests and retries failed requests with increasing pauses, so it never puts noticeable load on the server. It also saves every page as soon as it is processed and skips pages already saved, so an interrupted crawl continues where it stopped.

## 3.3 Why a Second Filter Is Needed

Selecting pages by slug has a hidden problem, caused by the missing accents. Vietnamese has many words that differ only in their accents, and once the accents are removed they become identical:

- `tieu-duong` is *tiểu đường* (diabetes), but also the beginning of *tiểu dương vật* (a urology question about a child);
- `dai-duong` is *đái đường* (diabetes), but also *đại dương* (ocean), as in *cá ngừ đại dương* (ocean tuna);
- `duong` also appears in *dưỡng ẩm* (moisturising) and *dương tính* (positive, of a test result).

These pages pass the slug test but have nothing to do with diabetes. So after downloading, every page is checked again on its real text, **with accents**, where the words can no longer be confused. The check counts how often the page mentions diabetes (*tiểu đường*, *đái tháo đường*, *ĐTĐ*—the "diabetes count") and how often it uses related terms (*đường huyết*, *insulin*, *HbA1c*, *biến chứng*, …—the "related count"). A page is kept if:

1. it is an official Vinmec **disease page**: Vinmec has already classified it as a diabetes page; or
2. its **title or summary** mentions diabetes and the text mentions it at least twice: the title is the author's own statement of what the article is about, so it is the strongest signal; or
3. its slug had a **strong** keyword and the text mentions diabetes at least three times: this removes the accent collisions, which mention diabetes zero times; or
4. its slug had only a **weak** keyword and the text mentions diabetes at least five times *and* uses related terms at least five times: an article "Insulin là gì?" that discusses the hormone thirty times but mentions diabetes once is about biology, not the disease.

The thresholds are deliberately loose where the evidence is strong (the title) and strict where the risk of a wrong page is high (a weak keyword). Table 3 shows the result, and Table 4 shows some of the rejected pages.

Table 3: Candidate pages kept and rejected by the relevance filter, by strength of the slug keyword.

| Slug keyword | Kept | Rejected | Candidates |
|---|---|---|---|
| strong | 934 | 10 | 944 |
| weak | 162 | 85 | 247 |
| **Total** | 1,096 | 95 | 1,191 |

Table 4: Examples of rejected pages, with their diabetes count and related count. The first four are accent collisions.

| Title | Slug keyword | Diabetes count | Related count |
|---|---|---|---|
| Trẻ 19 tháng đi tiểu dương vật không cử động cần làm gì? | strong | 0 | 0 |
| Vô tình hít phải Heroin nên đi thử nước tiểu dương tính có phải không? | strong | 0 | 0 |
| Kết quả xét nghiệm nước tiểu dương tính với Amphetamine và Methamphetamine có ý nghĩa gì? | strong | 0 | 0 |
| Thành phần dinh dưỡng trong cá ngừ đại dương | strong | 2 | 0 |
| Dầu hoa anh thảo dưỡng ẩm da có tốt không? | strong | 0 | 0 |
| Bài thuốc từ bí đao chữa bệnh tiểu đường | strong | 1 | 0 |

Only 10 of the 944 strong-keyword pages were rejected, and all of them are collisions such as *tiểu dương vật* and *đại dương*. Among the weak-keyword pages a third were rejected: they are about insulin, blood sugar or tests in general rather than about the disease. The filter is not perfect: the article *"Bài thuốc từ bí đao chữa bệnh tiểu đường"* is about diabetes but too short to reach the threshold. Losing a handful of short pages costs little; letting off-topic pages in would add false facts to the graph.

### 3.4 Parsing a Page

The crawler does not save the raw HTML of a page. Raw HTML contains menus, advertisements, booking forms, the footer and scripts; the article itself is a small part of it. The parser uses the structure of Vinmec's pages:

- **articles**: the body is the element with class `entry`; the title, publication date, author and summary come from the page's structured metadata (`application/ld+json`, the format search engines read);
- **disease pages**: each section is an element with class `item_detial_sick` containing a heading and a body.

The body is split into **sections**. For each section three things are saved: the **heading** it sits under, its **paragraphs** and its **bullet points** as separate items.

Keeping the heading is the most important design decision of the whole project. The heading tells us what kind of information the text below it contains: the bullet "Mệt mỏi" (fatigue) means something completely different under *Triệu chứng* (symptoms) than under *Tác dụng phụ của thuốc* (side effects of a drug). Section 4.3 builds the whole extraction on this.

The text at the end of articles that is not part of the article is cut off: the booking text (*"Để đặt lịch khám tại viện, Quý khách vui lòng bấm số HOTLINE..."*), the app advertisement and the "Xem thêm:" (see also) links. If it were kept, every article would "mention" the same booking words, and the related-article links would bring in words from other topics. Figure 2 shows the saved form of the type 2 disease page.

```
title: Tiểu đường tuýp 2: Nguyên nhân, triệu chứng, chẩn đoán và điều trị
url:   https://www.vinmec.com/vie/benh/tieu-duong-tuyp-2-3042     type: disease

## Tổng quan bệnh Tiểu đường tuýp 2            (4 paragraphs, 0 bullets)
## Nguyên nhân bệnh Tiểu đường tuýp 2          (1 paragraph,  0 bullets)
## Triệu chứng bệnh Tiểu đường tuýp 2          (3 paragraphs, 4 bullets)
     • Ăn nhiều
     • Uống nhiều (hay khát nước)
     • Tiểu nhiều (do đường trong nước tiểu cao, gây lợi niệu thẩm thấu)
     • Gầy nhiều (gầy sút cân): bệnh nhân tiểu đường tuýp 2 thường có ...
## Biến chứng cấp tính                         (1 paragraph,  0 bullets)
## Biến chứng mạn tính                         (0 paragraphs, 6 bullets)
     • Nhìn mờ (do biến chứng võng mạc, đục thủy tinh thể)
     • Tê bì dị cảm ở bàn chân (biến chứng thần kinh)
     ...
## Đối tượng nguy cơ bệnh Tiểu đường tuýp 2    ...
## Các biện pháp chẩn đoán bệnh Tiểu đường tuýp 2  ...
## Các biện pháp điều trị bệnh Tiểu đường tuýp 2   ...
```

Figure 2: The saved form of Vinmec's type 2 diabetes page: a list of sections, each with its heading, paragraphs and bullet points (abbreviated).

### 3.5 Data Volume

Table 5: Size of the collected corpus.

| Quantity | Value |
|---|---|
| Candidate URLs from the sitemap | 1,191 |
| Pages kept after the relevance filter | 1,096 |
| Official disease pages among them | 4 |
| Total words | 1,130,914 |
| Median words per page | 1,044 |
| Shortest / longest page (words) | 109 / 4,099 |
| Publication years | 2024–2026 |
| Size of the saved JSON pages | 9.4 MB |

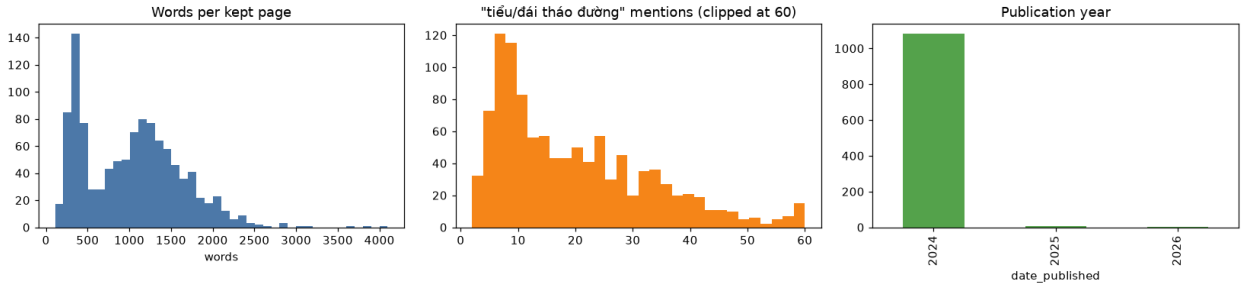



Figure 3: Left: words per kept page. Middle: how often each page mentions diabetes (clipped at 60). Right: publication year of the kept pages.

Figure 3 gives a picture of the corpus. A typical article has about a thousand words, long enough to cover several aspects of the disease under separate headings. Half of the kept pages mention diabetes 16 times or more, and only one in ten mentions it fewer than 6 times, which confirms that the filter kept pages about the disease rather than pages that mention it in passing. Almost all pages (1,083 of 1,092 with a date) carry a 2024 publication date. This is the date of Vinmec's re-publication of its archive rather than the date the article was written, so the real age of each article is unknown; this is discussed as a limitation in Section 10.

# 4 Knowledge Graph Design

## 4.1 Facts, Entities and Relations

A Knowledge Graph is a set of **triples**. Each triple states one fact in a fixed three-part form: the thing the fact is about (the **subject**), the kind of relationship (the **relation**) and the other thing (the **object**). The paragraph of the type 2 disease page that lists the classic symptoms and the chronic complications becomes facts such as:

| subject | relation | object |
|---|---|---|
| Đái tháo đường type 2 | HAS_SYMPTOM | Khát nhiều |
| Đái tháo đường type 2 | HAS_SYMPTOM | Tiểu nhiều |
| Đái tháo đường type 2 | HAS_COMPLICATION | Bệnh võng mạc |
| Bệnh võng mạc | AFFECTS_ORGAN | Mắt |

Why is this better than the paragraph? Because every fact has the same predictable form. A program does not need to understand Vietnamese grammar to use it; it only looks for facts whose subject is "type 2" and whose relation is `HAS_COMPLICATION`. The ambiguity of language is resolved once, when the fact is created, instead of every time a question is asked.

If every thing is drawn as a dot and every fact as an arrow, the facts form a network: a **graph**. The things are the **nodes** or **entities**, the facts are the **edges** or **relationships**, and each node also has a **type** (`Symptom`, `Drug`, …) so a program knows what kind of thing it is. Figure 4 shows seven hand-written facts as a graph. Questions become walks along the arrows: "which organ does type 2 damage?" starts at *type 2*, follows `HAS_COMPLICATION` to *Bệnh thận đái tháo đường* and then `AFFECTS_ORGAN` to *Thận*. No single fact contains the answer; combining two facts through the node in the middle does. This is what the lecture calls a **two-hop** query, as opposed to a **one-hop** query that only reads the arrows directly attached to one node.

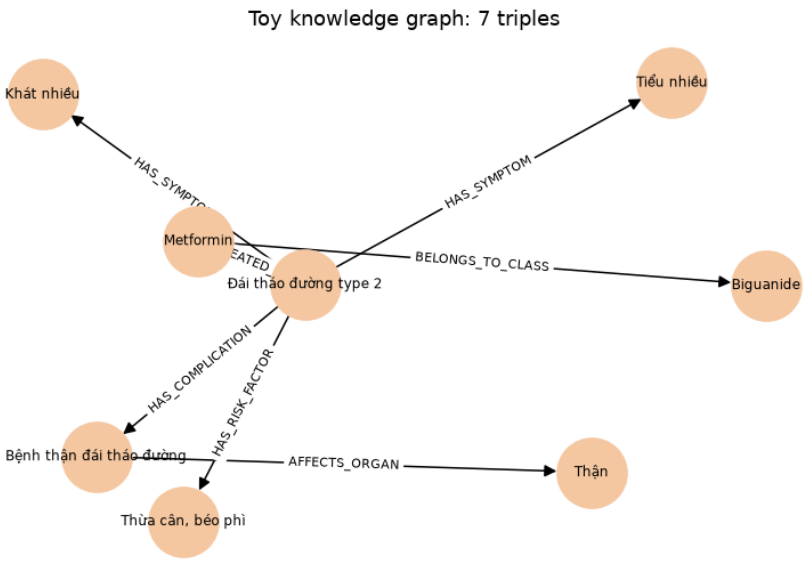



Figure 4: Seven hand-written diabetes facts drawn as a graph (from the notebook, Part 1). The organ *Thận* is two hops from *type 2*: no single fact connects them.

The real graph has the same shape but is built automatically from 1,096 articles. The hard part of the project is not storing the graph—it is extracting **correct** facts from free Vietnamese text without a person reading every article. The rest of this section explains how that is done.

## 4.2 Schema: Entity Types and Relation Types

The schema says which kinds of entities exist and which kinds of facts may connect them. It was designed from the questions in Table 2 and from the way Vinmec articles are organised.

**Entity types.** There are ten types (Table 6). Five of them correspond to the standard sections of a disease article: symptoms, risk factors, complications, tests and treatments. Treatments are split into `Drug` and `Lifestyle` because the questions about them differ ("which drug class?" only makes sense for drugs). `Food` is separate because diet is one of the most discussed topics. `DrugClass` and `Organ` are not extracted from text; they exist to group drugs and to locate complications in the body (Section 4.3.9). The five `DiabetesType` nodes are the subjects of almost all extracted facts.

Table 6: Entity types: number of vocabulary entries, aliases (ways of writing them), entities that ended up in the graph, and examples.

| Type | Vocabulary | Aliases | In graph | Examples |
|---|---|---|---|---|
| DiabetesType | – | – | 5 | Tiền đái tháo đường, Đái tháo đường, Đái tháo đường thai kỳ |
| Symptom | 16 | 56 | 16 | Buồn nôn, Chóng mặt, Da sạm (gai đen) |
| RiskFactor | 17 | 52 | 16 | Chế độ ăn nhiều đường, tinh bột, Căng thẳng, stress, Di truyền / tiền sử gia đình |
| Complication | 19 | 52 | 18 | Bàn chân đái tháo đường, Bệnh nha chu, Bệnh thần kinh ngoại biên |
| Test | 10 | 35 | 10 | C-peptide, HbA1c, Kháng thể kháng đảo tụy |
| Drug | 22 | 42 | 22 | Acarbose, Canagliflozin, Dapagliflozin |
| DrugClass | – | – | 9 | Biguanide, Meglitinide, Nhóm Insulin |
| Food | 16 | 66 | 16 | Cá béo, Các loại hạt, Các loại đậu |
| Lifestyle | 6 | 17 | 6 | Bỏ thuốc lá, Chăm sóc bàn chân, Giảm cân |
| Organ | – | – | 9 | Bàn chân, Dạ dày, Mắt |
| **Total** | 106 | 320 | 127 | |

Almost every vocabulary entry was found in the articles with enough support to enter the graph; the one risk

factor and one complication that are missing are discussed in Section 9.

**Relation types.** Table 7 lists the twelve relation types with the entity types they connect and how many facts each one has in the graph. The `source` column separates facts extracted from articles (`vinmec`) from general classifications added by hand (`curated`).

Table 7: Relation types in the graph.

| Relation | From → to | Source | Facts |
|---|---|---|---|
| HAS_COMPLICATION | DiabetesType → Complication | vinmec | 52 |
| TREATED_BY | DiabetesType → Drug | vinmec | 46 |
| HAS_SYMPTOM | DiabetesType → Symptom | vinmec | 45 |
| HAS_RISK_FACTOR | DiabetesType → RiskFactor | vinmec | 42 |
| RECOMMENDS_FOOD | DiabetesType → Food | vinmec | 35 |
| DIAGNOSED_BY | DiabetesType → Test | vinmec | 30 |
| AVOIDS_FOOD | DiabetesType → Food | vinmec | 26 |
| BELONGS_TO_CLASS | Drug → DrugClass | curated | 22 |
| MANAGED_BY | DiabetesType → Lifestyle | vinmec | 19 |
| AFFECTS_ORGAN | Complication → Organ | curated | 12 |
| HAS_SIDE_EFFECT | Drug → Complication | vinmec | 5 |
| SUBTYPE_OF | DiabetesType → DiabetesType | curated | 4 |
| HAS_SIDE_EFFECT | Drug → Symptom | vinmec | 1 |

The direction of every relation is a modelling choice: `HAS_SYMPTOM` points from the disease to the symptom, but it could equally be written `SYMPTOM_OF` in the other direction. The convention used here is that facts point *away* from the diabetes types and then onward to classifications. Section 6.5 shows that this choice matters when computing graph measures. Figure 5 draws the schema: every arrow is a kind of question the graph can answer, and a missing arrow (for example from `Food` to `Complication`) is a question it cannot answer.

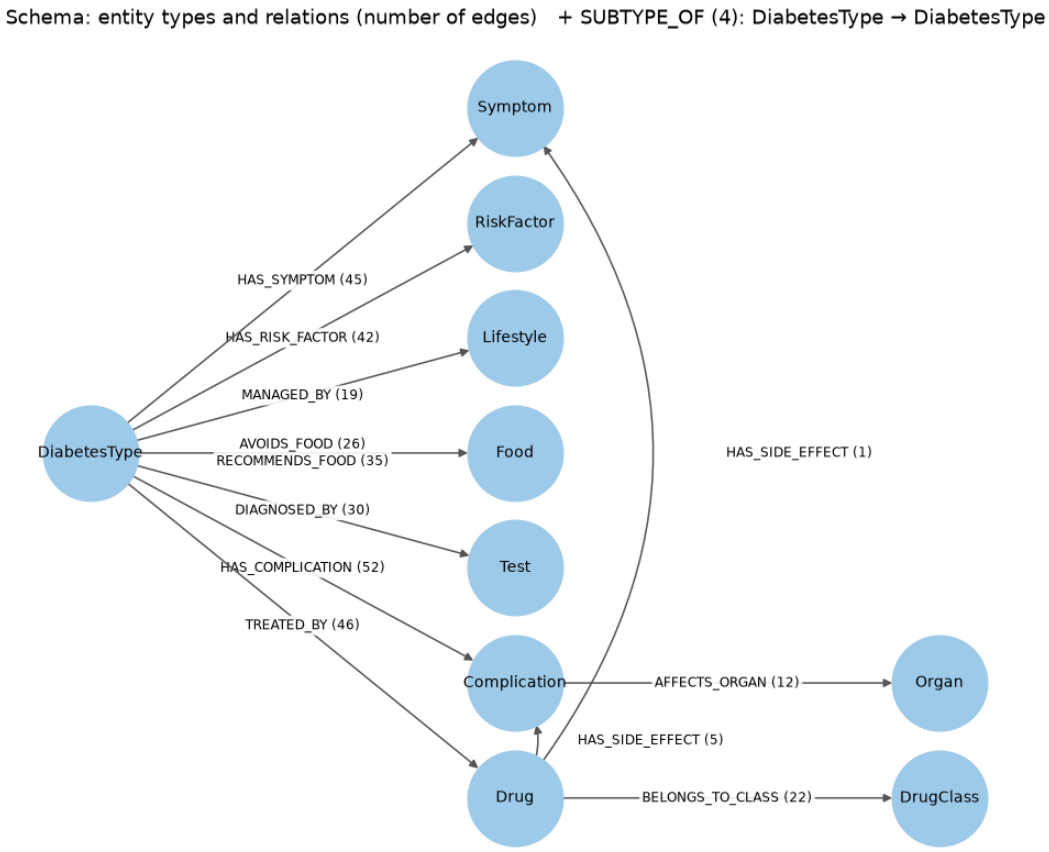



Figure 5: The schema of the graph as stored in Neo4j: entity types (circles) and the relations between them, with the number of facts of each kind. Diabetes types on the left, what the articles say about them in the middle, hand-added classifications on the right.

## 4.3 Extraction Method

### 4.3.1 Why not let each bullet point become a node

The simplest approach would be to make every bullet under *Triệu chứng* a Symptom node. On the type 2 page that gives nodes such as "Ăn nhiều" and "Uống nhiều (hay khát nước)", which looks acceptable. But a bullet under *Điều trị* on the type 1 page reads:

> *"Kiểm soát đường huyết: đối với tuýp 1 thì kiểm soát bằng insulin ngoại sinh là chủ yếu. Dùng theo phác đồ, tuân thủ giờ tiêm và liều lượng tránh bị tụt đường huyết. Các loại insulin gồm có…"*

That is sixty words. As a node it is useless: no other article will ever write the same sixty words, so it never connects to anything and no question ever matches it. Even short bullets differ: one article writes *uống nhiều*, another *khát nước*, another *hay khát*. As separate nodes, the single fact "diabetes causes thirst" would be split into weak pieces that never meet.

So instead of copying text into the graph, the project decides **in advance** which concepts the graph may contain, gives each one a clean name, and searches the text for any of the ways it can be written. Extraction then answers three questions for every section of every page:

1. **Which diabetes is the page about?** This is the subject of the facts.
2. **What kind of information may this section contain?** This is decided by the section heading.
3. **Which known concepts does the section mention?** These are the objects of the facts.

Each fact found on a page is one **vote** from that page; the votes of all pages are counted and a fact enters the graph only when enough different pages agree. Figure 6 shows the process on a real section.

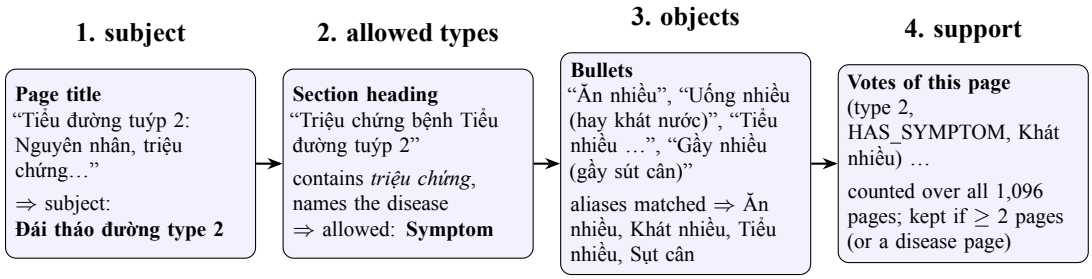



Figure 6: Extraction of symptom facts from the symptom section of Vinmec's type 2 disease page.

### 4.3.2 The vocabulary and entity normalisation

The vocabulary (`kg/vocab.py`) lists 106 concepts. Each has a clean **canonical name** that becomes the node, a **type**, and a list of **aliases**: the different ways the articles write it (320 aliases in total). A few entries:

```
VOCAB = {
    "Khát nhiều":     ("Symptom", ["khát nhiều", "uống nhiều", "khát nước", "hay
    ↪  khát", "khô miệng"]),
    "Tê bì tay chân": ("Symptom", ["tê bì", "tê tay", "tê chân", "dị cảm", "ngứa
    ↪  ran", "kiến bò"]),
    "Kháng insulin": ("RiskFactor", ["kháng insulin", "đề kháng insulin"]),
    "Bệnh võng mạc": ("Complication", ["võng mạc", "biến chứng mắt", "mù lòa"]),
    "HbA1c":          ("Test", ["hba1c", "hemoglobin a1c", "a1c"]),
    "Metformin":      ("Drug", ["metformin", "glucophage"]),
    "Empagliflozin": ("Drug", ["empagliflozin", "jardiance"]),
    "Rau xanh":       ("Food", ["rau xanh", "rau lá xanh", "rau củ", "bông cải",
    ↪  "súp lơ"]),
    ...
}
```

Mapping many surface forms to one node is called **entity normalisation**. It is what makes evidence add up: when one article says *uống nhiều* and another *khát nước*, both vote for the same fact *(diabetes, HAS_SYMPTOM, Khát nhiều)*, and the graph knows that two articles agree. Brand names are included as aliases (*Glucophage*, *Diamicron*, *Januvia*, *Jardiance*, *Lantus*) because Vietnamese articles often name drugs by brand.

The vocabulary was written by hand from the topics that appear in the articles: the classic symptoms, the risk factors listed on Vinmec's pages, the acute and chronic complications, the standard tests, the drugs used in Vietnam, and the foods and lifestyle measures that diet articles discuss. Writing it by hand has an obvious cost—a concept that is not in the list can never be found—and an obvious benefit: every node in the graph is clean, meaningful and correctly typed. The cost is analysed in Section 10.

### 4.3.3 Finding an alias in the text

All aliases of an entity are combined into one regular expression. For *Tiểu nhiều*:

```
(?<!\w)(đi tiểu thường xuyên|tiểu thường xuyên|đi tiểu nhiều|tiểu nhiều|tiểu
↪  đêm)(?!\w)
```

Three details of this pattern matter.

**The alias must be a whole word.** Vietnamese writes syllables separated by spaces, and many aliases are one short syllable that also appears inside other words. Without protection, the fruit alias *lê* (pear) would match inside *lên* (as in *lên cân*, gaining weight) and the alias *bia* (beer) inside *biên*. The parts `(?<!\w)` and `(?!\w)` require that the characters before and after the alias are not letters: “bệnh nhân lên cân” contains no food, “ăn quả lê” does.

**Longer aliases are tried first.** If "tiểu thường xuyên" were tried before "đi tiểu thường xuyên", the shorter alias would match first and the more precise wording would never be used.

**Accents are kept.** Text is lower-cased so that "HbA1C" and "hba1c" match, but accents are never removed: removing them would bring back exactly the collisions seen in the URLs (*tiểu đường* and *tiểu dương*).

What word boundaries cannot prevent is an alias that is itself a whole word with a second meaning: the fruit alias *táo* (apple) matches *táo bón* (constipation). The section rule below is what keeps such errors rare, because a food alias only counts inside diet sections.

#### 4.3.4 The section rule: the heading decides what may be extracted

Finding a vocabulary word somewhere in an article does not mean that the article states a fact about it. Real examples from the corpus:

- *mệt mỏi* (fatigue) appears in a diet article: "ăn sáng đầy đủ giúp bạn đỡ mệt mỏi". This is not a statement that fatigue is a symptom of diabetes.
- *tăng huyết áp* (high blood pressure) appears in drug paragraphs as a condition the drug should not be combined with.
- almost every article mentions *insulin* while explaining how blood sugar works, not because insulin is the treatment being discussed.

If every match counted, the graph would claim that diabetes has dozens of "symptoms" that are merely words that happened to appear. So a match only counts when the **section heading** says that this section is about that kind of information. This works because health articles—especially on a hospital website—are organised by exactly these headings. Table 8 gives the rule.

Table 8: The section rule: keywords in a heading and the entity types that may be extracted below it.

| Heading contains | Allowed types | Relation produced |
|---|---|---|
| triệu chứng, dấu hiệu, biểu hiện, nhận biết | Symptom | HAS_SYMPTOM |
| nguyên nhân, nguy cơ, đối tượng, yếu tố, ai dễ | RiskFactor | HAS_RISK_FACTOR |
| biến chứng, nguy hiểm, hậu quả, ảnh hưởng | Complication | HAS_COMPLICATION |
| chẩn đoán, xét nghiệm, phát hiện, chỉ số, kiểm tra, theo dõi | Test | DIAGNOSED_BY |
| điều trị, thuốc, insulin, kiểm soát | Drug, Lifestyle | TREATED_BY, MANAGED_BY |
| phòng ngừa, phòng tránh, ngăn ngừa, lối sống, lưu ý | Lifestyle | MANAGED_BY |
| ăn, thực phẩm, dinh dưỡng, thực đơn, món, uống, kiêng, tránh | Food | RECOMMENDS_FOOD / AVOIDS_FOOD |

A section whose heading contains none of these keywords—for example *Tổng quan* (overview)—produces no facts. This is intentional: an overview mentions everything briefly, and the project prefers to miss a fact rather than invent one.

**An extra check for symptoms and risk factors.** The first version of the rule had a weakness that the quality check in Section 7.2 revealed. A heading can contain *nguyên nhân* or *nguy cơ* and still not be about diabetes itself:

- *"Một số nguyên nhân gây ngứa bàn tay và bàn chân"* lists the causes of **itching**;
- *"Yếu tố nguy cơ gây tai biến mạch máu não ở người tiểu đường"* lists the risk factors for a **stroke** in people who already have diabetes;
- *"Làm gì để giảm thiểu nguy cơ tim mạch…"* is advice on **reducing** heart risk.

Reading the bullets under these headings as risk factors *of diabetes* produced wrong facts. The reason is that a symptom or a risk factor always belongs to *something*, and the heading names what that something is. So, for these two types only, the heading must also pass the function `about_disease()`:

```
DISEASE_TERMS = ["tiểu đường", "đái tháo đường", "đtđ", "tuýp", "type", "típ",
↪  "bệnh"]
OTHER_TOPIC   = ["biến chứng", "thận", "võng mạc", "mắt", "thần kinh", "bàn
↪  chân", "tim mạch",
                 "ngứa", "gout", "da", "răng", "nhiễm toan", "ceton", "hạ đường
                 ↪  huyết",
                 "thai nhi", "trẻ sinh ra", "tai biến", "đột quỵ", "mạch vành",
                 ↪  "hôn mê"]
REDUCE_VERBS  = ["giảm", "phòng", "tránh", "ngừa", "hạn chế", "kiểm soát"]

def about_disease(heading):
    """Generic heading ("Triệu chứng", "Ai dễ bị?") or one that names diabetes,
    ↪  and no other topic."""
    h = re.sub(r"^[\d\.\s]+", "", heading.lower())          # drop numbering
    ↪  such as "2.1."
    if any(t in h for t in OTHER_TOPIC) or any(v + " nguy cơ" in h for v in
    ↪  REDUCE_VERBS):
        return False
    return any(t in h for t in DISEASE_TERMS) or len(h.split()) <= 5

def allowed_labels(heading):
    h = heading.lower()
    labels = set()
    for keys, labs in SECTION_RULES:                          # the rule of
    ↪  Table 7
        if any(k in h for k in keys):
            labels |= labs
    if not about_disease(heading):
        labels -= {"Symptom", "RiskFactor"}
    return labels
```

A heading must therefore either name diabetes or be short and generic (“Triệu chứng”, “Ai dễ bị?”—there is nothing else it could be about), must not name another topic, and must not be about reducing a risk. Table 9 shows the rule applied to real headings from the corpus.

Table 9: The section rule applied to real headings.

| Heading | About diabetes itself? | Allowed types |
|---|---|---|
| Triệu chứng bệnh Tiểu đường tuýp 2 | yes | Symptom |
| Đối tượng nguy cơ bệnh Tiểu đường tuýp 1 | yes | RiskFactor |
| 3. Ai dễ bị tiền tiểu đường? | yes | RiskFactor |
| Các biện pháp điều trị bệnh Tiểu đường tuýp 1 | yes | Drug, Lifestyle |
| 2.2. Một số thực phẩm mẹ bầu mắc bệnh tiểu đường thai kỳ cần tránh | yes | Food |
| 3. Một số nguyên nhân gây ngứa bàn tay và bàn chân | no | – |
| 2. Yếu tố nguy cơ gây tai biến mạch máu não ở người tiểu đường | no | – |
| 4. Làm gì để giảm thiểu nguy cơ tim mạch ở bệnh nhân tiểu đường? | no | – |
| Biến chứng mạn tính | yes | Complication |

#### 4.3.5 Which diabetes is the subject?

Diabetes is not one disease. Type 1 is an autoimmune disease in which the body destroys its own insulin-producing cells, usually starting young. Type 2 develops slowly in adults and is linked to weight and lifestyle. Gestational diabetes appears during pregnancy. Prediabetes is the stage before type 2. Their symptoms, causes and treatments overlap but are not identical: “autoimmune destruction of beta cells” is a cause of type

1 only, “pre-eclampsia” a complication of gestational diabetes only. Attaching every fact to one “diabetes” node would lose these differences.

Each page is therefore assigned a subject from its **title**, because the title is the author’s own short statement of what the article is about; counting words in the body is less reliable, since an article about type 2 often mentions type 1 for comparison. The title is checked for the words of each type in turn (*tuýp 1 / type 1 / típ 1*, *tuýp 2 / type 2*, *thai kỳ / mang thai / mẹ bầu*, *tiền đái tháo đường / tiền tiểu đường*); if none is found, the subject is diabetes in general.

This is a simplification with a visible consequence (Figure 7): most articles do not name a type in the title, so the general node receives most of the facts, even though many of those articles are in fact about type 2. It was accepted deliberately: placing a type 2 fact on the general node makes the graph less specific, while placing it on the type 1 node would make it false.

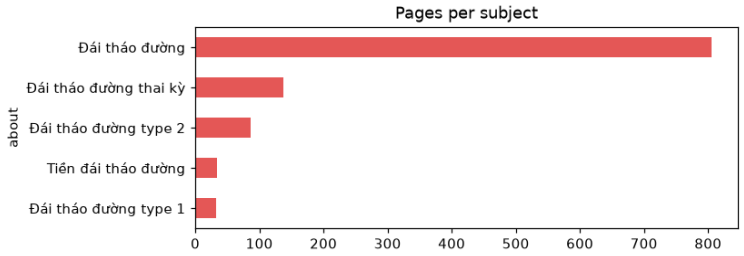



Figure 7: Number of pages assigned to each subject. Most article titles do not name a specific type.

### 4.3.6 Counting evidence: why one page is not enough

Every rule above can still go wrong on a single page: an unusual heading, a symptom mentioned in passing, a bullet that lists something for comparison. If every page were trusted, every one of these mistakes would become a fact. The defence is to require **agreement between independent pages**. For every fact the extractor keeps the *set* of pages that voted for it—a set, so a page that repeats “khát nhiều” five times counts once. The size of the set is the fact’s **support**.

A small example with three pages about type 2:

| Page | Facts it votes for |
|---|---|
| A | type 2 HAS_SYMPTOM Khát nhiều; type 2 HAS_SYMPTOM Mệt mỏi |
| B | type 2 HAS_SYMPTOM Khát nhiều |
| C | type 2 HAS_SYMPTOM Khát nhiều; type 2 HAS_SYMPTOM Run tay |

*Khát nhiều* is stated by three pages (support 3) and is kept; *Mệt mỏi* and *Run tay* are stated by one page each and are dropped. The idea is the same as checking a claim against a second source. The rule has two exceptions, each with a reason:

- a fact from an official Vinmec **disease page** is accepted with support 1: these are short, structured summaries reviewed by the hospital, with exactly the standard headings, so extraction from them is the most reliable;
- a **side effect** needs support 3, because that extraction works on single sentences and is noisier (below).

```
for (h, rel, t), urls in support.items():                    # urls = set of pages
↪  voting for the fact
    need = MIN_SUPPORT_SIDE_EFFECT if rel == "HAS_SIDE_EFFECT" else MIN_SUPPORT
    ↪  # 3 or 2
    if len(urls) >= need or (h, rel, t) in from_disease:
        triples.append((h, rel, t, len(urls), "vinmec", "
        ↪  ".join(sorted(urls)[:5])))
```

The support number is not thrown away after this filter. It is stored on the fact in Neo4j, so queries can rank facts by how well documented they are, and the website lets the reader hide facts below a chosen level of evidence.

### 4.3.7 Foods: recommended or to avoid

For foods the graph has two relations, `RECOMMENDS_FOOD` and `AVOIDS_FOOD`. The first version decided between them from the words in the sentence or heading: *nên ăn* (should eat) meant recommended; *kiêng*, *tránh*, *hạn chế*, *không nên* meant avoid. It failed, and the reason is instructive: diet articles mix both kinds of advice in the same place. A heading such as *"Thực phẩm mẹ bầu cần tránh và nên ăn"* contains both, and a single sentence often says "hạn chế đồ ngọt, thay vào đó nên ăn rau xanh". The output contained nonsense such as *gestational diabetes AVOIDS green vegetables*.

The final rule treats whether a food is good or bad for diabetes as a **property of the food**, written once in the vocabulary: sweets, soft drinks, white rice and refined starch, very sweet fruit (durian, ripe mango, longan, jackfruit), alcohol, fried food and processed meat are marked "avoid"; the other foods are "recommended". The text then only tells us that the food is discussed in a diet section for that type of diabetes. This matches standard nutrition advice and what the articles say in almost all cases; the cost is that nuanced advice ("fruit is fine in moderate amounts") cannot be represented.

### 4.3.8 Drug side effects

Side effects are scattered across drug sections, complication sections and general articles, so the section rule cannot find them. They are extracted **inside one sentence** instead: a sentence produces a side-effect fact only if it contains a drug, a symptom or complication, and a word that signals cause—*tác dụng phụ* (side effect), *tác dụng không mong muốn* (unwanted effect) or *gây* (causes):

> "Insulin có thể **gây hạ đường huyết** nếu tiêm quá liều" ⇒ (Insulin, HAS_SIDE_EFFECT, Hạ đường huyết)

Sentences containing *thiếu* (lack of) or *không đủ* (not enough) are skipped. The sentence "*thiếu* insulin gây nhiễm toan ceton" has a drug, a complication and the word *gây*, but it says that a *lack* of insulin causes ketoacidosis—the opposite of a side effect of taking insulin. Without the exception the graph would warn that insulin causes the very condition it treats. Section 9 shows that this filter catches most but not all such sentences.

### 4.3.9 Background facts added by hand

Some knowledge is not something articles state under a heading, but general medical classification that articles take for granted: Metformin belongs to the biguanide class; diabetic retinopathy affects the eye; the diagnostic threshold of HbA1c is 6.5%. These facts are added by hand and labelled `source = curated`, so they can always be told apart from extracted facts:

- each drug and its **drug class** (22 facts: Metformin → Biguanide, Empagliflozin → Ức chế SGLT2, …);
- each complication and the **organ** it affects (12 facts);
- each type of diabetes as a **subtype** of diabetes in general (4 facts);
- the diagnostic **thresholds** stored as a property of four test nodes, from the Vietnamese Ministry of Health 2020 guideline and the American Diabetes Association criteria that Vinmec's pages also quote: HbA1c $\geq$ 6.5%; fasting plasma glucose $\geq$ 7.0 mmol/L; 2-hour glucose in the tolerance test $\geq$ 11.1 mmol/L; random glucose $\geq$ 11.1 mmol/L with symptoms.

These few facts make the most interesting questions possible. Without "retinopathy affects the eye", the graph could say that type 2 causes retinopathy but could not answer "which organs does type 2 damage?".

## 4.4 Output Files

The extractor writes four CSV files, each with a clear purpose:

Table 10: Files produced by the extractor (`dataset/processed/`).

| File | One row is | Why it exists |
|---|---|---|
| `entities.csv` | an entity: name, type, threshold | becomes the nodes |
| `triples.csv` | a fact: subject, relation, object, support, source, up to five example URLs | becomes the relationships |
| `articles.csv` | a crawled page: URL, title, page type, date, subject | every fact can be traced back to its articles |
| `mentions.csv` | a pair (page, entity) for every entity mentioned anywhere on the page | a looser signal of what each article talks about |

The facts file uses the `subject, relation, object` form of the lecture's Exercise 1, with three additional columns:

```
subject,relation,object,support,source,example_urls
Đái tháo đường type 2,HAS_SYMPTOM,Khát
↪  nhiều,4,vinmec,https://www.vinmec.com/vie/bai-viet/...
Đái tháo đường type
↪  2,TREATED_BY,Metformin,11,vinmec,https://www.vinmec.com/vie/bai-viet/...
Metformin,BELONGS_TO_CLASS,Biguanide,0,curated,
Bệnh võng mạc,AFFECTS_ORGAN,Mắt,0,curated,
```

Table 11 shows the best-supported extracted fact of each relation type.

Table 11: The best-supported extracted fact of each relation type.

| Subject | Relation | Object | Support |
|---|---|---|---|
| Đái tháo đường | TREATED_BY | Insulin | 143 |
| Đái tháo đường | MANAGED_BY | Tập thể dục đều đặn | 114 |
| Đái tháo đường | HAS_COMPLICATION | Bệnh tim mạch | 56 |
| Đái tháo đường | RECOMMENDS_FOOD | Ngũ cốc nguyên hạt | 54 |
| Đái tháo đường | AVOIDS_FOOD | Rượu bia | 48 |
| Đái tháo đường | DIAGNOSED_BY | HbA1c | 42 |
| Insulin | HAS_SIDE_EFFECT | Hạ đường huyết | 37 |
| Đái tháo đường | HAS_RISK_FACTOR | Thừa cân, béo phì | 27 |
| Đái tháo đường | HAS_SYMPTOM | Mệt mỏi | 25 |

## 4.5 Meeting the Lecture's Requirements

Table 12: Lecture requirements (Mini Project 3, Exercise 1) and what the graph achieves.

| Requirement | Lecture minimum | This project |
|---|---|---|
| Entities | $\geq 50$ (Mini Project 3), $\geq 20$ (Exercise 1) | 127 |
| Relationships | $\geq 100$ (Mini Project 3), $\geq 30$ (Exercise 1) | 339 |
| Relationship types | $\geq 5$ | 12 |
| Entity types | $\geq 3$ | 10 |
| Cypher queries | $\geq 5$ | 15 |
| Triples stored as CSV | `subject, relation, object` | `triples.csv` |
| One-hop and two-hop queries | required | queries 12–14, `graph_facts` |
| Graph visualisation | required | NetworkX figures, website |

# 5 System Implementation

## 5.1 Project Structure

The system is written in Python (crawler, extractor, loader, API, notebook) and JavaScript (website). Table 13 lists every part and its role. Every stage reads its input from files or from the database and can be re-run on its own.

Table 13: Project structure of `graph_01/`.

| Path | Role |
|---|---|
| `kg/crawl.py` | sitemap → candidate URLs → download → relevance filter → `dataset/raw/pages/*.json` and `index.csv`; resumable |
| `kg/vocab.py` | vocabulary (106 entities, 320 aliases), section rules, food polarity, curated classes, organs and thresholds |
| `kg/extract.py` | pages → votes → support filter → `dataset/processed/*.csv` |
| `kg/load.py` | CSV files → Neo4j (wipes and reloads; idempotent) |
| `kg/retriever.py` | `graph_facts(entity, hops)`: bounded graph retrieval |
| `kg/visualize.py` | NetworkX drawings of the graph |
| `kg/evaluate.py` | draws the random sample of facts for the quality check |
| `kg/queries.cypher` | the 15 example queries, ready for Neo4j Browser |
| `notebook/01_diabetes_kg.ipynb` | explains and runs every stage, with all figures |
| `api/` | FastAPI service: read-only Cypher queries behind a small REST interface |
| `web/` | React website for exploring the graph |
| `evaluation/manual_audit.csv` | the 50 sampled facts with the manual judgement |
| `report/` | this report and its plan |

## 5.2 Storing the Graph in Neo4j

### 5.2.1 Why a graph database

The NetworkX graph of the lecture's first examples lives in the memory of a Python program: when the program ends it is gone, and every program that needs it must rebuild it. A real system needs the knowledge stored permanently, shared by several programs (the notebook, the website, Cypher queries) and answered quickly.

One could store the triples in an ordinary relational database as a table with three columns. But questions about a graph are questions about *paths*: start here, follow these arrows, then those. In a table database every step of a path is a join of the table with itself; the queries become long and hard to read and get slower as the table grows. A **graph database** stores each node with direct references to its neighbours, so following an arrow is a direct jump rather than a search. **Neo4j** is the most widely used graph database, and its query language **Cypher** describes paths with an ASCII-art syntax that looks like a drawing of the path. Neo4j 5 runs locally in a Docker container, with its data stored in the project folder.

### 5.2.2 Data model

```
(:Entity:Symptom {name})                          every entity: label Entity + its
↪  own type
(:Entity:Test {name, threshold})
(:DiabetesType)-[:HAS_SYMPTOM {support, source, example_urls}]->(:Symptom)
(:Article {url, title, page_type, date})-[:ABOUT]->(:DiabetesType)
(:Article)-[:MENTIONS]->(:Entity)
```

Three decisions in this model deserve an explanation.

**Every entity has two labels.** The type label (`Symptom`) lets a query ask for "all symptoms". The shared `Entity` label lets one rule cover all entities—every entity name must be unique, so there can never be two

different “Metformin” nodes—and lets a query such as “everything connected to Metformin” be written without knowing in advance the type of each neighbour.

**The evidence is stored on the relationships.** `support`, `source` and `example_urls` are properties of each fact. One query can therefore return the answer, how well documented it is and which articles state it. If the evidence were stored elsewhere, every answer would need a second lookup.

**The articles are nodes too.** Each of the 1,096 articles is an `Article` node, connected by `ABOUT` to its diabetes type and by `MENTIONS` to every entity it mentions anywhere (10,083 links). This keeps the connection between the clean structured facts and the original text, and answers “where does this come from?”. Because article nodes and `MENTIONS` links are not facts about diabetes, all queries about the knowledge itself restrict themselves to `Entity` nodes.

### 5.2.3 Loading

```
CREATE CONSTRAINT entity_name IF NOT EXISTS FOR (e:Entity) REQUIRE e.name IS
 ↪  UNIQUE;

UNWIND $rows AS r
MERGE (e:Entity {name: r.name}) SET e:Symptom;          -- one query per entity
 ↪  type

UNWIND $rows AS r
MATCH (h:Entity {name: r.subject}), (t:Entity {name: r.object})
MERGE (h)-[x:HAS_SYMPTOM]->(t)                           -- one query per
 ↪  relation type
SET x.support = r.support, x.source = r.source, x.example_urls = r.example_urls;
```

- **`MERGE` instead of `CREATE`.** `CREATE` always adds a new node or relationship, even when an identical one exists. `MERGE` first looks for it and creates it only if it is missing. The loader can therefore be run any number of times without duplicating anything.
- **`UNWIND` sends a whole batch.** Each request to the database has a fixed cost. Sending the 10,083 mention links one by one would mean 10,083 round trips; `UNWIND` sends a list of rows in one request and lets the database loop over it. The whole graph loads in a few seconds.
- **The uniqueness constraint is also an index.** To enforce unique names, Neo4j builds an index on `name`, so every query that starts from “the node called Metformin” jumps directly to it instead of checking all nodes.
- **Relation types are written into the query text.** Cypher does not allow the type of a relationship (or a label) to be a parameter, so the loader groups the facts by relation and builds one query per type from the fixed list of twelve.

After loading, the database contains 1,223 nodes (127 entities and 1,096 articles) and 11,518 relationships (339 facts, 10,083 mentions and 1,096 `ABOUT` links).

## 5.3 Querying with Cypher

A Cypher query describes the *shape* of a path: round brackets for nodes, square brackets inside an arrow for relationships.

```
MATCH (t:DiabetesType {name: 'Đái tháo đường type
 ↪  2'})-[r:HAS_SYMPTOM]->(s:Symptom)
RETURN s.name, r.support
```

Read from left to right: find a `DiabetesType` node called "type 2", follow an outgoing `HAS_SYMPTOM` arrow to a `Symptom` node, return the symptom's name and the arrow's support. Neo4j uses the name index to jump to the "type 2" node and then looks only at the arrows attached to it; it never scans the rest of the database. The work grows with the number of arrows followed, not with the size of the database. The fifteen example queries of `kg/queries.cypher` and their results are presented in Section 7.1.

## 5.4 Graph Retrieval

In a complete question-answering system (the lecture's Knowledge Graph + RAG architecture), the graph's job is to hand a language model a small set of relevant, reliable facts. `kg/retriever.py` implements the lecture's `graph_facts` on Neo4j:

```
ONE_HOP = """
MATCH (e:Entity {name: $name})-[r]-(n:Entity)
RETURN startNode(r).name AS s, type(r) AS rel, endNode(r).name AS o, r.support
 ↪  AS support
ORDER BY support DESC"""
TWO_HOP = """
MATCH (e:Entity {name: $name})-[r1]-(m:Entity)-[r2]-(n:Entity) WHERE n <> e
RETURN startNode(r2).name AS s, type(r2) AS rel, endNode(r2).name AS o,
 ↪  r2.support AS support
ORDER BY support DESC"""

def graph_facts(driver, entity, hops=1, limit=30):
    facts, seen = [], set()
    with driver.session() as s:
        for q in [ONE_HOP] + ([TWO_HOP] if hops >= 2 else []):
            for r in s.run(q, name=entity):
                key = (r["s"], r["rel"], r["o"])
                if key not in seen:
                    seen.add(key)
                    facts.append(f"{r['s']} {r['rel']} {r['o']}.")
    return facts[:limit]
```

It starts at the entity named in the question, collects the facts within one (or two) steps, turns each back into a short sentence, sorts by support and returns at most 30.

```
$ python kg/retriever.py "Hạ đường huyết"
Đái tháo đường HAS_COMPLICATION Hạ đường huyết.
Insulin HAS_SIDE_EFFECT Hạ đường huyết.
Đái tháo đường thai kỳ HAS_COMPLICATION Hạ đường huyết.
Metformin HAS_SIDE_EFFECT Hạ đường huyết.
Đái tháo đường type 2 HAS_COMPLICATION Hạ đường huyết.
```

Why limit the number of hops and facts? In a graph almost everything is connected to everything within a few steps, especially through hub nodes. One step from *Hạ đường huyết* (hypoglycaemia) gives the 5 precise facts above; two steps give **230**, because one first-step neighbour is the general diabetes node, which is connected to almost the whole graph. Giving 230 facts to a language model for a question about hypoglycaemia would bury the relevant ones in noise. This is the problem the lecture calls "graph retrieval too broad" (§23.4), and the solution used here is the one it recommends: start from the entity in the question, stay within one or two hops, and rank and cut the list.

## 5.5 The Website

### 5.5.1 Architecture

Cypher is powerful but not something a typical reader can use. The website makes the graph usable by anyone. It has two parts:

- an **API** (`api/`, Python, FastAPI) that receives simple HTTP requests, runs fixed Cypher queries against Neo4j and returns JSON;
- a **web client** (`web/`, React 18 and Vite, hand-written CSS, the same stack as the course's earlier web applications) that the user sees in the browser.




**Why an API in between** rather than letting the browser talk to Neo4j directly? Three reasons. Security: the database password never leaves the server, and the browser can only run the eight fixed, read-only queries the API offers—it cannot send arbitrary Cypher. Stability: the website depends on a small documented interface, not on the database schema, so the schema can change without breaking it. Simplicity: every request is a plain URL such as `/entities/Metformin`, which can be tested in a browser and is documented automatically by FastAPI at `/docs`.

### 5.5.2 API

Table 14: API endpoints.

| Endpoint | Returns |
|---|---|
| `GET /stats` | Number of entities per type, facts per relation, articles |
| `GET /entities?q=&type=` | Search by name (prefix matches first, then by number of connections) |
| `GET /entities/{name}?min_support=` | One entity: type, threshold, article count and all its facts |
| `GET /evidence?subject=&relation=&object=` | The source articles stored on one fact |
| `GET /entities/{name}/articles` | Latest articles that mention the entity |
| `POST /symptom-check` | Diabetes types ranked by matching symptoms |
| `GET /paths?source=&target=` | Shortest chains of facts between two entities (up to 4 steps) |
| `GET /health` | Is the database reachable? (used for the offline banner) |

Every query is parameterised: user input is passed as a parameter (`$name`), never pasted into the query text, so a search for an unusual string cannot change the query. Every response has a declared shape (a Pydantic model), which both validates the output and documents it. Errors are translated into plain messages: an unknown entity gives `404 No entity named …`, a stopped database gives `503 The Neo4j database is not reachable`, which the website shows as a banner telling the user what to start. The entity endpoint, for example:

```
@app.get("/entities/{name}", response_model=EntityDetail)
def entity(name: str, min_support: int = Query(0, ge=0)):
    rows = db.read(f"""
        MATCH (e:Entity {{name: $name}})
        RETURN e.name AS name, {TYPE.format(v='e')} AS type, e.threshold AS
        ↪  threshold,
               COUNT {{ (:Article)-[:MENTIONS|ABOUT]->(e) }} AS mentioned_in""",
               ↪  name=name)
    if not rows:
        raise HTTPException(404, f"No entity named '{name}'.")
    facts = db.read(f"""
        MATCH (e:Entity {{name: $name}})-[r]-(n:Entity)
        WHERE r.source = 'curated' OR r.support >= $min_support
        WITH startNode(r) AS s, r, endNode(r) AS o
        RETURN s.name AS subject, {TYPE.format(v='s')} AS subject_type, type(r)
        ↪  AS relation,
               o.name AS object, {TYPE.format(v='o')} AS object_type,
```

```
            coalesce(r.support, 0) AS support, r.source AS source
        ORDER BY support DESC LIMIT $cap""", name=name, min_support=min_support,
        ↪  cap=config.MAX_FACTS)
    return {**rows[0], "facts": facts}
```

The path finder uses Neo4j's built-in `allShortestPaths` over entity nodes only (articles are excluded, otherwise every pair of entities would be connected through some article that mentions both), limited to four steps. For each step it also returns whether the fact points forward or backward along the path, so the website can phrase it correctly ("*is a risk factor for*" rather than "*has the risk factor*").

### 5.5.3 Web client

The client has five pages, each with its own address so that any view can be bookmarked or shared (for example `#/entity/Metformin` or `#/paths?from=…&to=…`):

1. **Home**: a search box with suggestions as one types, the size of the graph, entry points for each diabetes type, and a short "how this graph was built" section;
2. **Entity**: all facts about one entity, grouped under plain-language headings, each with its support and a *Sources* button that lists the articles stating it; a drawing of the entity's neighbourhood; the latest articles that mention it;
3. **Diabetes types**: the same page for each type, in tabs, to compare them;
4. **Symptom checker**: choose symptoms and see which types of diabetes the articles associate with them;
5. **Connections**: choose two entities and see the shortest chains of facts between them.

Several design decisions follow from the evaluation of the graph rather than from appearance:

- **Evidence is always visible.** Each fact shows how many articles state it, and its sources are one click away. Hand-added facts are marked "reference".
- **The reader chooses the level of evidence.** A selector hides facts stated by fewer than 2, 3, 5 or 10 articles (default 2). This exposes the precision/recall trade-off of Section 9.3 to the user instead of fixing it for them.
- **Relations are written in plain words.** `HAS_COMPLICATION` seen from the disease is shown as "Complications", seen from the complication as "Complication of"; `TREATED_BY` seen from a drug becomes "Used to treat".
- **The medical disclaimer is explicit** where it matters, on the symptom checker: the result shows what the articles say and is not a diagnosis.
- **Every state is handled**: loading indicators, empty results, an unknown entity, and a stopped API or database (a banner with the command to start it). The search box is a standard accessible combobox (arrow keys, Enter, Escape), and the layout adapts to phone screens.

Section 8 walks through the website with screenshots.

## 6 Visualisation

A graph of 127 entities and 339 facts cannot be understood from tables alone. This section shows it from several angles, from the whole network down to single neighbourhoods, and explains what each picture shows. All drawings are made with NetworkX from the same CSV files that were loaded into Neo4j, so they show exactly the same facts.

## 6.1 Size of the Graph

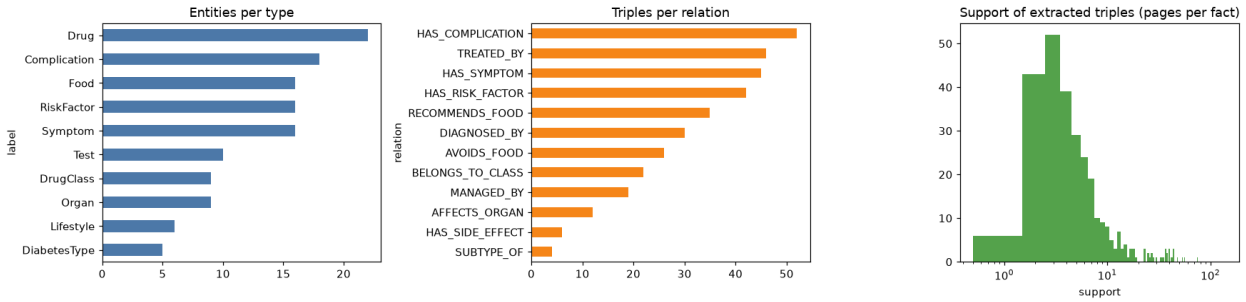



Figure 8: Left: entities per type. Middle: facts per relation type. Right: support of the extracted facts (how many articles state each fact), on a logarithmic scale.

The left chart of Figure 8 shows that drugs, complications, risk factors, symptoms and foods each have between 16 and 22 entities—roughly the size of the vocabulary for each type, which means almost every concept in the vocabulary was found with enough support.

The right chart is the most informative. Most facts are stated by a handful of articles (the median fact has support 5), but a few are repeated by dozens: "diabetes is treated with insulin" by 143 articles, "managed by regular exercise" by 114, "can cause cardiovascular disease" by 56. This long-tailed shape is typical of a health website. The core message (exercise, diet, insulin, the main complications) is repeated in almost every article, while specific facts—which drug class a newer medicine belongs to, which complications are specific to pregnancy—are written in only a few. Support is therefore not only a filter against noise but also a measure of how central a fact is to the way Vinmec writes about diabetes.

## 6.2 The Whole Graph

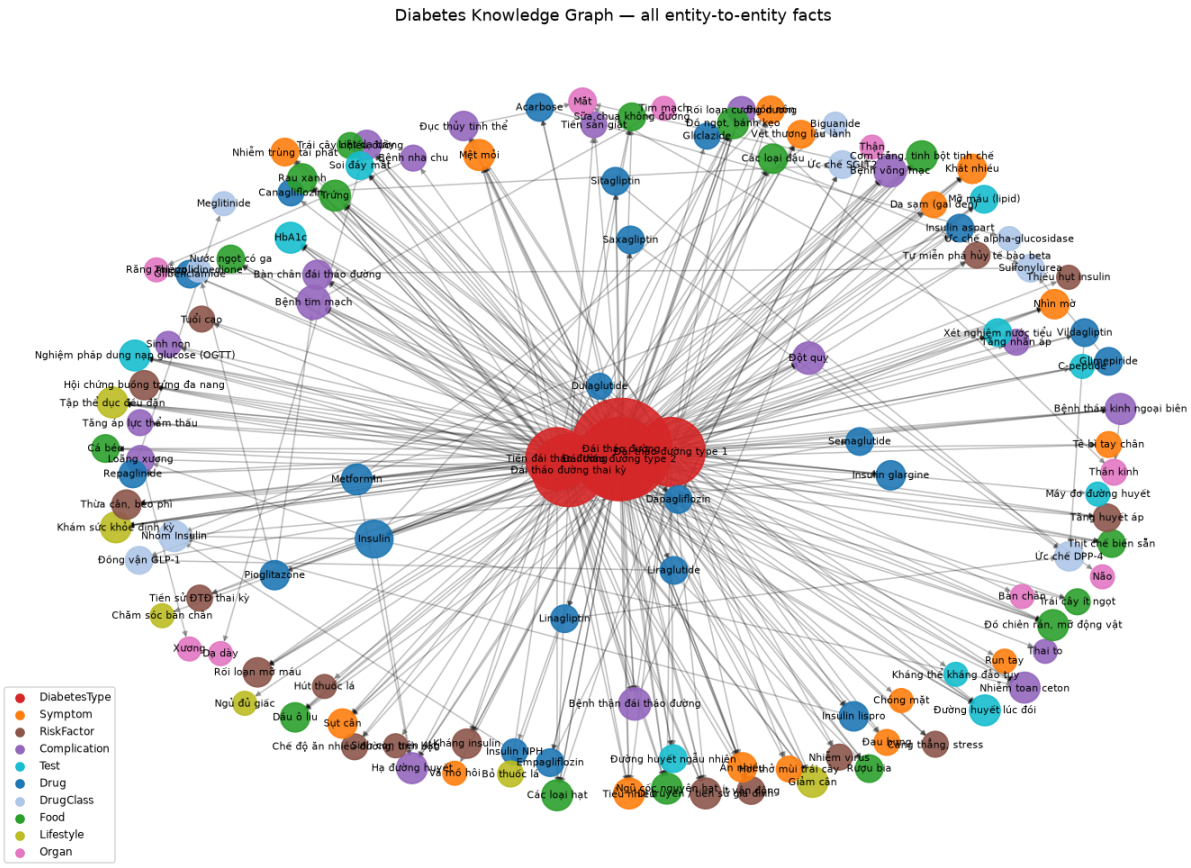



Figure 9: The whole Knowledge Graph: all 339 entity-to-entity facts. Colour = entity type; size = number of facts the entity takes part in.

Figure 9 shows the overall structure. The five diabetes-type nodes are the large hubs near the centre, because almost every extracted fact starts at one of them. Symptoms, complications, drugs, foods and tests are arranged around them, many connected to several types at once (thirst is a symptom of all of them). The hand-added facts form small chains at the edge: drug → drug class, complication → organ.

The graph is **sparse**. Of all the arrows that could exist between 127 entities, only about 2% are present: each entity is connected to a few others, not to everything. This is typical of real knowledge, and it is what makes graph queries fast—following the arrows of one node touches a small part of the graph.

## 6.3 Focused Views

The whole picture shows the structure but is too dense to read in detail. Three smaller views are easier to read.

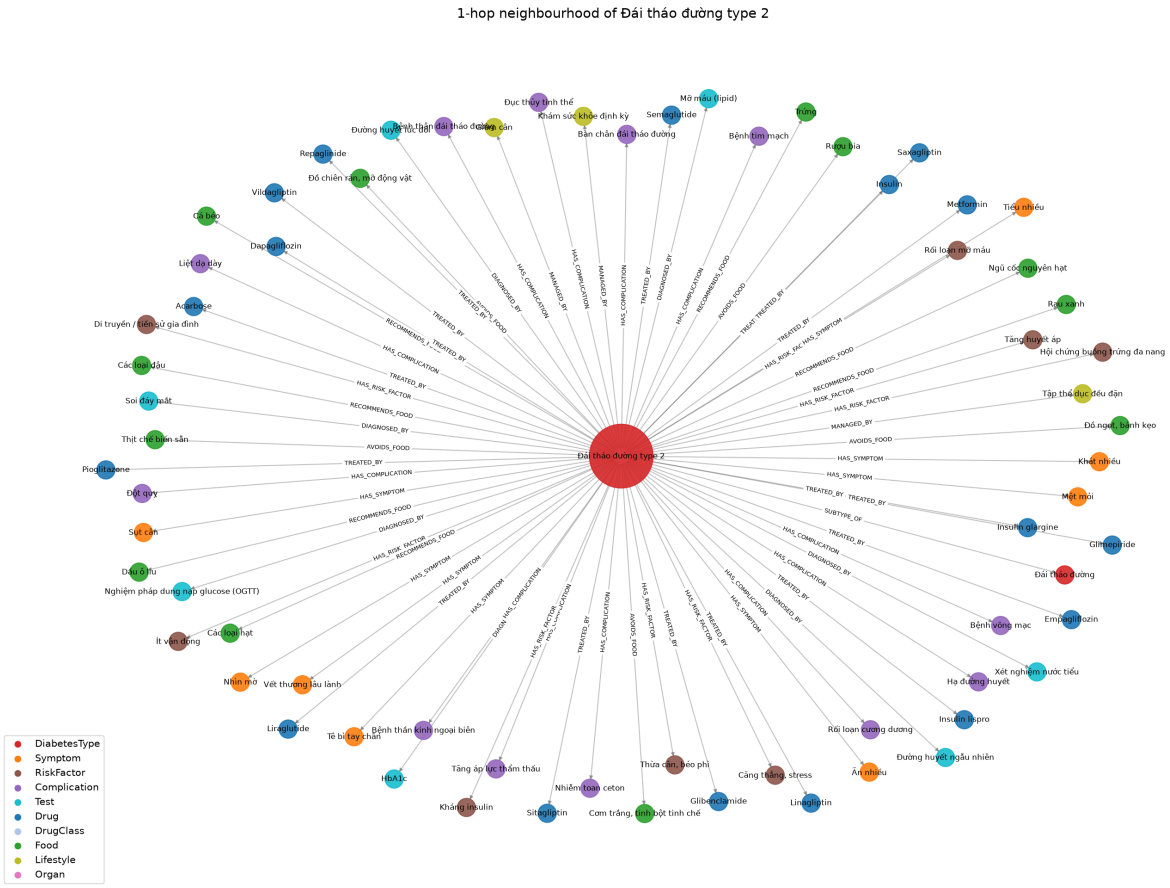



Figure 10: One-hop neighbourhood of *Đái tháo đường type 2*: every fact that touches the type 2 node, with its relation.

Figure 10 is everything the graph knows directly about type 2 diabetes: 17 drugs, 12 complications, 8 risk factors, 8 symptoms, 7 tests, 12 foods and 3 lifestyle measures. It is the visual form of a one-hop query.

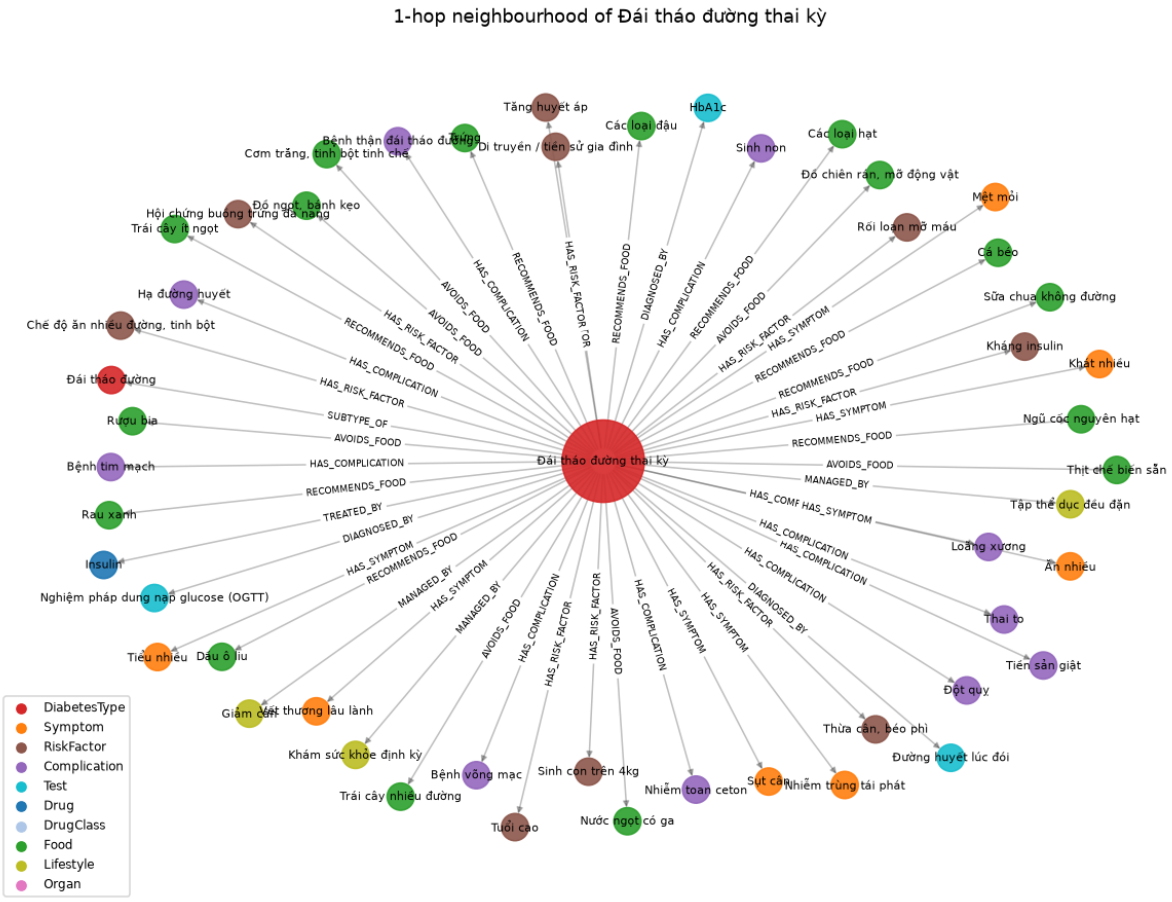



Figure 11: One-hop neighbourhood of *Đái tháo đường thai kỳ* (gestational diabetes).

Figure 11 shows gestational diabetes. Compared with type 2 it has a single drug, insulin (most oral drugs are avoided during pregnancy, so insulin is the standard treatment) and its own complications: pre-eclampsia, premature birth and a very large baby. The graph has captured this difference from the articles without being told.

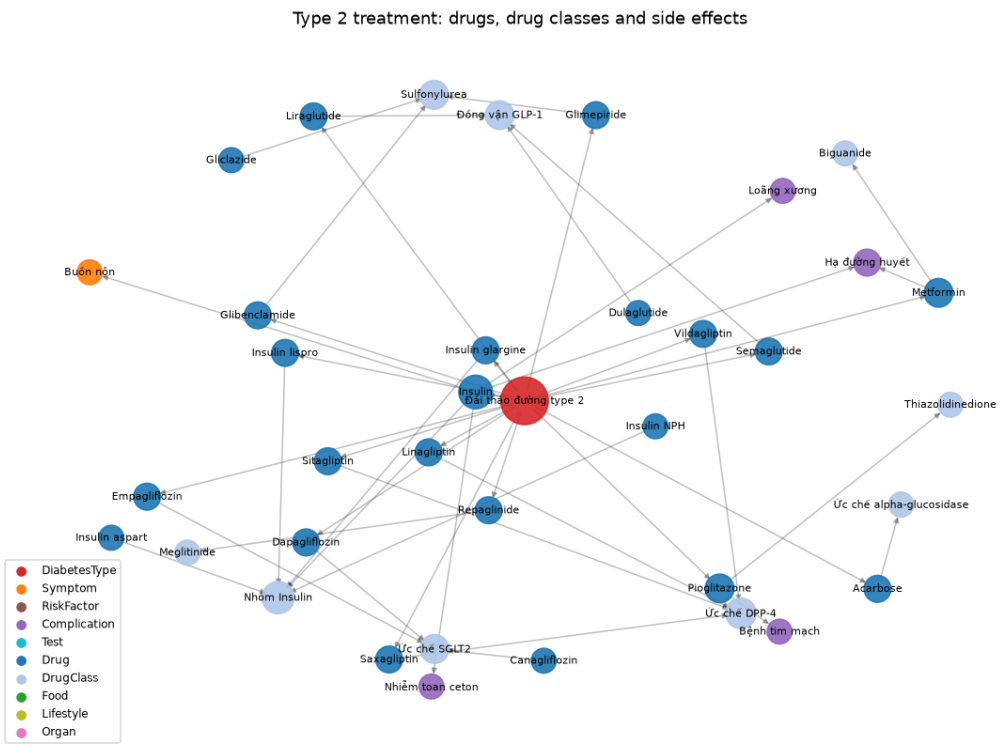



Figure 12: Type 2 treatment: the drugs used for type 2, their classes (hand-added) and their extracted side effects.

Figure 12 combines extracted and hand-added facts. The class arrows make the drug families appear as clusters—three DPP-4 inhibitors around their class, three SGLT2 inhibitors around theirs, the insulin family—which is how doctors think of these drugs. The side-effect arrows show the most discussed risk of treatment, hypoglycaemia, linked to insulin and Metformin.

## 6.4 Comparing the Diabetes Types

If assigning a separate subject to each type (Section 4.3) works, the types should show different facts. The heatmaps in Figure 13 show, for each complication and each symptom, how many articles support the fact for each type (darker = more articles; empty = no fact).




Figure 13: Supporting articles per diabetes type for the most common complications (left) and symptoms (right).

Two things are visible. First, the general diabetes column is by far the darkest: most articles do not name a type in their title, so their facts go to the general node. Second, gestational diabetes has its own clear pattern. Its strongest complications are hypoglycaemia (of the mother on treatment and of the newborn), pre-eclampsia, premature birth and a large baby—pregnancy complications that the other types do not have—while type 1 and type 2 share the classic long-term complications of the eyes, kidneys, nerves and heart. This is medically correct and confirms that the subject rule captures real differences.

### 6.5 Which Entities Are the Most Important?

"Important" can mean different things for a node in a graph, and two common measures give very different answers here. Understanding why is a good way to understand the graph itself.

**Degree** is the number of facts an entity takes part in. It answers "how many facts involve this entity?" and looks only at the node's own arrows.

**PageRank**, the method Google originally used to rank web pages, says that a node is important if *important* nodes point to it. It is easiest to understand as a walk: a person starts on a random node and, at each step, either follows one of the outgoing arrows of the current node at random or, occasionally (15% of the time), jumps to a completely random node. After a very long walk, a node's PageRank is the share of time spent on it. Nodes that many paths lead into are visited often.

**What happens in this graph.** All arrows point away from the diabetes types: type → drug → drug class, type → complication → organ. A walker following the arrows drifts down these chains and arrives at their ends—drug classes and organs—which have no outgoing arrows, so the walker stays there until the next random jump. Directed PageRank therefore ranks *Nhóm Insulin*, *Ức chế DPP-4* and *Mắt* (eye) highest. They are not what the articles are mainly about; they are simply where many paths end.

The direction of the arrows is our own modelling choice ("type 2 HAS_SYMPTOM thirst" could be written "thirst SYMPTOM_OF type 2"), and a measure whose answer flips when a relation is renamed does not measure what we want. PageRank is therefore also computed **ignoring direction**, as if every arrow could be walked both ways. Then an entity is important if it is well connected to other well-connected entities. This ranking makes sense: the diabetes types first, then insulin, cardiovascular disease, stroke, diabetic kidney disease, retinopathy and Metformin—the treatments and complications that tie the whole graph together (Figure 14). The lesson is that every graph measure answers a specific question, and one must check that it is the question one wants to ask.

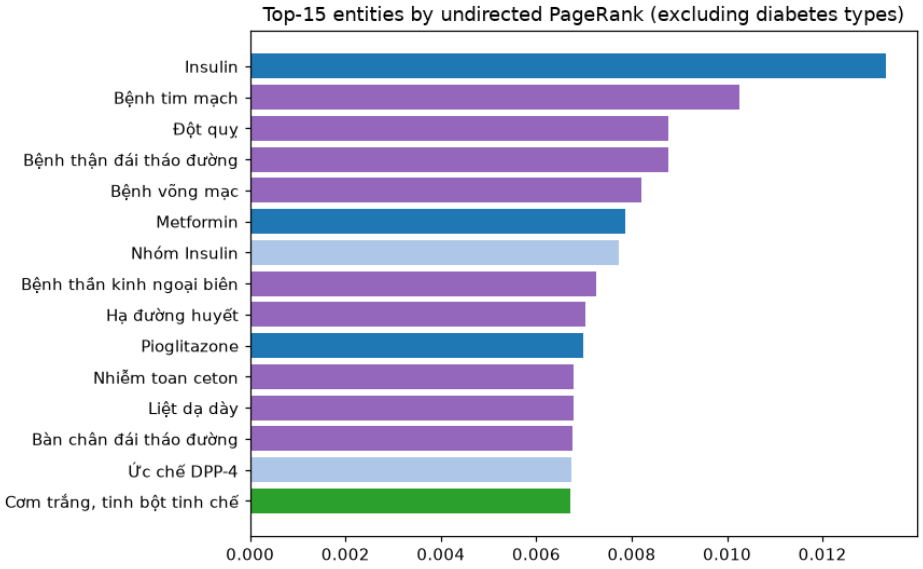



Figure 14: The 15 most central entities by undirected PageRank (diabetes types excluded). Colour = entity type.

### 6.6 What Vinmec Writes About Most

A third view of importance does not use the facts at all. It counts in how many of the 1,096 articles each entity is mentioned anywhere (the `MENTIONS` links). This measures what the website *talks about*, which differs from how connected an entity is among the facts: an entity can be mentioned in many articles in passing—for example while explaining how blood sugar works—while appearing in few facts under a proper heading.

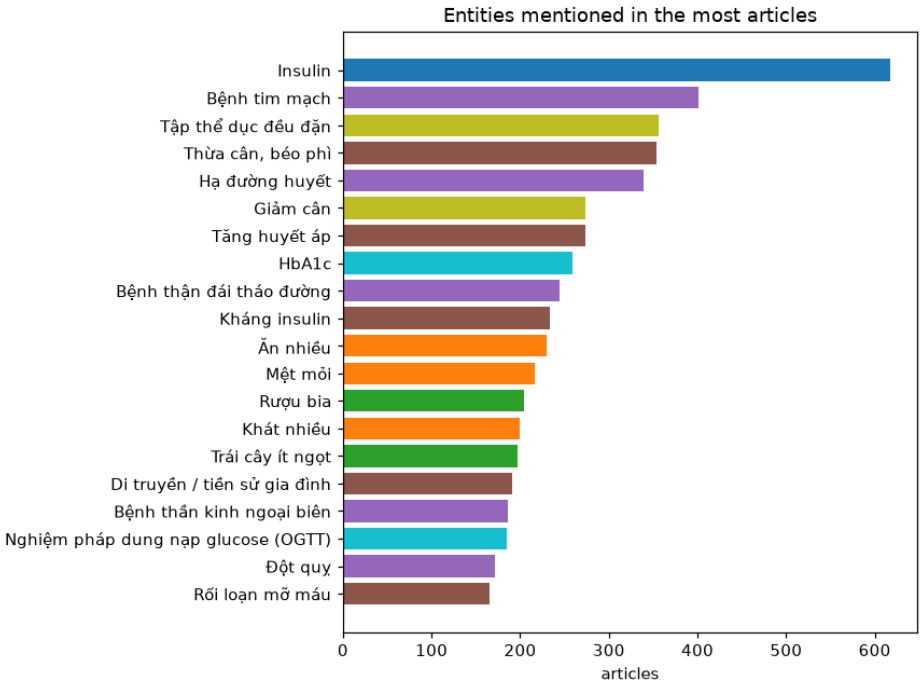



Figure 15: Entities mentioned in the largest number of articles.

## 7 Experimental Evaluation

The lecture insists that an intelligent system must be evaluated systematically, not judged by whether it "gives an answer". This section evaluates the graph in two ways. First, **what it can answer**: the example questions of Table 2 are run as Cypher queries and their results are checked for sense. Second, **how correct its facts are**: a random sample of extracted facts is checked by hand against the source articles.

### 7.1 Query Results

The queries below are taken from `kg/queries.cypher`, which contains fifteen queries in total (the lecture requires at least five). Each is shown with its result, produced directly from the database, and a short interpretation.

#### 7.1.1 One-hop: everything about Metformin

```
MATCH (e:Entity {name: 'Metformin'})-[r]-(n:Entity)
RETURN startNode(r).name AS subject, type(r) AS relation, endNode(r).name AS
 ↪  object,
       r.support AS support, r.source AS source
ORDER BY support DESC
```

Table 15: One-hop neighbourhood of Metformin.

| Subject | Relation | Object | Support | Source |
|---|---|---|---|---|
| Đái tháo đường | TREATED_BY | Metformin | 37 | vinmec |
| Đái tháo đường type 2 | TREATED_BY | Metformin | 11 | vinmec |
| Tiền đái tháo đường | TREATED_BY | Metformin | 8 | vinmec |
| Metformin | HAS_SIDE_EFFECT | Hạ đường huyết | 5 | vinmec |
| Metformin | BELONGS_TO_CLASS | Biguanide | 0 | curated |

The pattern `-[r]-` has no arrowhead and matches relationships in both directions, so the result contains facts that point *to* Metformin (diabetes *is treated with* Metformin) and the fact that points *from* it (Metformin *belongs to* the biguanide class). For "what do we know about X" both directions are needed. The support column shows how solid each fact is: 37 articles say that diabetes is treated with Metformin, 11 name type 2 specifically, 8 mention it for prediabetes (where it is used to delay the onset of type 2), and 5 mention hypoglycaemia as a risk. The drug class has support 0 because it was added by hand.

### 7.1.2 Two-hop: which organs can type 2 diabetes damage?

```
MATCH (t:DiabetesType {name: 'Đái tháo đường type 2'})-[r:HAS_COMPLICATION]->(c)
      -[:AFFECTS_ORGAN]->(o:Organ)
RETURN o.name AS organ, collect(c.name) AS via_complications, sum(r.support) AS
 ↪  evidence
ORDER BY evidence DESC
```

Table 16: Organs reached from type 2 diabetes in two hops.

| Organ | Via complications | Evidence (articles) |
|---|---|---|
| Thần kinh | Rối loạn cương dương, Bệnh thần kinh ngoại biên | 9 |
| Mắt | Bệnh võng mạc, Đục thủy tinh thể | 8 |
| Tim mạch | Bệnh tim mạch | 8 |
| Não | Đột quỵ | 6 |
| Thận | Bệnh thận đái tháo đường | 5 |
| Bàn chân | Bàn chân đái tháo đường | 3 |
| Dạ dày | Liệt dạ dày | 1 |

No article states "type 2 diabetes damages the nerves" as a fact in this form. The answer exists only because two kinds of facts meet at the complication nodes: an extracted one (type 2 *can cause* peripheral neuropathy) and a hand-added one (peripheral neuropathy *affects* the nerves). `collect()` groups the complications leading to the same organ, so each organ appears once with its reasons. The result reproduces the textbook list of organs damaged by diabetes: nerves, eyes, heart and blood vessels, brain (stroke), kidneys, feet and stomach.

### 7.1.3 Comparing type 1 and type 2 symptoms

```
MATCH (s:Symptom)
OPTIONAL MATCH (:DiabetesType {name: 'Đái tháo đường type
 ↪  1'})-[a:HAS_SYMPTOM]->(s)
OPTIONAL MATCH (:DiabetesType {name: 'Đái tháo đường type
 ↪  2'})-[b:HAS_SYMPTOM]->(s)
WITH s, a, b WHERE a IS NOT NULL OR b IS NOT NULL
RETURN s.name, a.support AS type1, b.support AS type2
```

Table 17: Symptoms of type 1 and type 2 diabetes, with the number of supporting articles for each type.

| Symptom | Type 1 | Type 2 | Found in |
|---|---|---|---|
| Khát nhiều | 7 | 4 | both |
| Mệt mỏi | 6 | 3 | both |
| Nhìn mờ | 5 | 3 | both |
| Sụt cân | 7 | 2 | both |
| Tiểu nhiều | 6 | 5 | both |
| Ăn nhiều | 6 | 4 | both |
| Buồn nôn | 4 | – | type 1 only |
| Hơi thở mùi trái cây | 3 | – | type 1 only |
| Nhiễm trùng tái phát | 5 | – | type 1 only |
| Đau bụng | 5 | – | type 1 only |
| Tê bì tay chân | – | 3 | type 2 only |
| Vết thương lâu lành | – | 4 | type 2 only |

The six symptoms shared by both types—thirst, frequent urination, eating a lot, weight loss, fatigue and blurred vision—are the classic signs of high blood sugar itself, whatever its cause, so they should be shared. Most of the type-1-only symptoms—nausea, abdominal pain, fruity-smelling breath—are the signs of **ketoacidosis**, the dangerous acid build-up that happens when the body has almost no insulin, which is typical of type 1 at the time it is discovered. The type-2-only symptoms—slow-healing wounds and numb hands and feet—appear after years of moderately high blood sugar, which matches the slow, often unnoticed course of type 2. The graph reproduces a real medical difference without having been told about it. (Recurrent infections appear only for type 1 here; they are a general sign of high blood sugar that the type 2 articles simply did not list under a symptom heading.)

### 7.1.4 A symptom checker

```
WITH ['Khát nhiều', 'Tiểu nhiều', 'Sụt cân', 'Nhìn mờ'] AS input
MATCH (t:DiabetesType)-[r:HAS_SYMPTOM]->(s:Symptom) WHERE s.name IN input
RETURN t.name AS type, count(s) AS matched, collect(s.name) AS symptoms,
       sum(r.support) AS evidence
ORDER BY matched DESC, evidence DESC
```

Table 18: Diabetes types ranked for the symptoms {thirst, frequent urination, weight loss, blurred vision}.

| Type | Matched | Symptoms | Evidence |
|---|---|---|---|
| Đái tháo đường | 4 | Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều | 67 |
| Đái tháo đường type 1 | 4 | Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều | 25 |
| Đái tháo đường type 2 | 4 | Khát nhiều, Nhìn mờ, Sụt cân, Tiểu nhiều | 14 |
| Đái tháo đường thai kỳ | 3 | Khát nhiều, Sụt cân, Tiểu nhiều | 16 |
| Tiền đái tháo đường | 2 | Nhìn mờ, Tiểu nhiều | 7 |

The types are ranked by how many of the given symptoms they have; ties are broken by the total number of articles supporting the matching facts, that is, by how strongly the articles associate the type with these symptoms. General diabetes, type 1 and type 2 match all four; general diabetes comes first only because most articles are general (Section 4.3). The result shows both the use and the limit of the graph: it tells which types the articles associate with these symptoms and how strongly, but the classic symptoms are shared by all types, so it cannot distinguish them. A diagnosis requires blood tests—which is exactly what the next query shows.

### 7.1.5 Diagnosis: tests and thresholds

```
MATCH (t:DiabetesType)-[r:DIAGNOSED_BY]->(x:Test)
```

```
RETURN x.name AS test, x.threshold AS threshold, sum(r.support) AS evidence
ORDER BY evidence DESC
```

Table 19: Diagnostic tests, their thresholds (hand-added node property) and the number of supporting articles (extracted).

| Test | Threshold | Evidence |
|---|---|---|
| HbA1c | $\geq$ 6.5% | 67 |
| Nghiệm pháp dung nạp glucose (OGTT) | 2 giờ $\geq$ 11.1 mmol/L (200 mg/dL) | 59 |
| Đường huyết lúc đói | $\geq$ 7.0 mmol/L (126 mg/dL) | 51 |
| Xét nghiệm nước tiểu | – | 14 |
| Soi đáy mắt | – | 11 |
| Đường huyết ngẫu nhiên | $\geq$ 11.1 mmol/L kèm triệu chứng | 11 |
| Máy đo đường huyết | – | 10 |
| Mỡ máu (lipid) | – | 7 |
| C-peptide | – | 2 |
| Kháng thể kháng đảo tụy | – | 1 |

The answer combines the two kinds of knowledge in the graph. The link "diabetes is diagnosed with HbA1c" was extracted from the articles; the threshold "6.5% or more" is a property of the test node, added from the official guideline. The three tests with the most evidence—HbA1c, the glucose tolerance test and fasting glucose—are exactly the three standard diagnostic criteria. Tests with little evidence are specialised: the islet antibody test and C-peptide are used to tell type 1 from type 2. The graph links the antibody test to type 1 (from the type 1 disease page) and C-peptide only to diabetes in general, because the two articles that mention it do not name a type in their title.

### 7.1.6 Diet for gestational diabetes

Table 20: Foods discussed in diet sections of gestational-diabetes articles.

| Advice | Food | Articles |
|---|---|---|
| AVOIDS_FOOD | Đồ ngọt, bánh kẹo | 14 |
| AVOIDS_FOOD | Nước ngọt có ga | 7 |
| AVOIDS_FOOD | Thịt chế biến sẵn | 7 |
| AVOIDS_FOOD | Rượu bia | 6 |
| AVOIDS_FOOD | Đồ chiên rán, mỡ động vật | 6 |
| AVOIDS_FOOD | Trái cây nhiều đường | 4 |
| AVOIDS_FOOD | Cơm trắng, tinh bột tinh chế | 2 |
| RECOMMENDS_FOOD | Rau xanh | 13 |
| RECOMMENDS_FOOD | Trứng | 12 |
| RECOMMENDS_FOOD | Ngũ cốc nguyên hạt | 10 |
| RECOMMENDS_FOOD | Các loại đậu | 8 |
| RECOMMENDS_FOOD | Sữa chua không đường | 7 |
| RECOMMENDS_FOOD | Các loại hạt | 6 |
| RECOMMENDS_FOOD | Trái cây ít ngọt | 5 |
| RECOMMENDS_FOOD | Cá béo | 2 |
| RECOMMENDS_FOOD | Dầu ô liu | 2 |

Sweets are the most discussed food to avoid and green vegetables, eggs and whole grains the most recommended, consistent with the usual advice for pregnant women with diabetes. Remember that the split between "recommended" and "avoid" comes from the fixed food list (Section 4.3); what the articles contribute is *which* foods are discussed for this type and how often.

### 7.1.7 Provenance: where does a fact come from?

```
MATCH (:DiabetesType {name: 'Đái tháo đường type 2'})-[r:HAS_COMPLICATION]->
      (:Complication {name: 'Bệnh thận đái tháo đường'})
UNWIND r.example_urls AS url
MATCH (a:Article {url: url})
RETURN a.title
```

Table 21: Articles stating that type 2 diabetes can cause diabetic kidney disease.

| Article title |
|---|
| Bệnh tiểu đường type 2 có nguy hiểm không? |
| Bệnh tiểu đường type 2: Nguyên nhân, biến chứng và phòng bệnh |
| Các biến chứng nguy hiểm của bệnh đái tháo đường type 2 |
| Cách tự nhiên điều trị bệnh tiểu đường type 2 với những thực phẩm hàng ngày |
| Triệu chứng sớm của đái tháo đường type 2 |

Every fact stores up to five example article URLs; joining them with the article nodes returns the titles of real Vinmec articles that state the fact, which anyone can open and read. This is the property that makes a Knowledge Graph trustworthy for a health topic: a language model can say something fluent and wrong with no way to check it, while here every answer can be traced to its source.

### 7.1.8 Co-occurrence through articles

Table 22: Pairs of complications mentioned in the largest number of the same articles.

| Complication | Complication | Articles |
|---|---|---|
| Bệnh tim mạch | Hạ đường huyết | 170 |
| Bệnh thận đái tháo đường | Bệnh tim mạch | 163 |
| Bệnh tim mạch | Đột quỵ | 139 |
| Bệnh thận đái tháo đường | Hạ đường huyết | 116 |
| Bệnh thần kinh ngoại biên | Bệnh tim mạch | 110 |
| Bệnh thận đái tháo đường | Bệnh võng mạc | 107 |

The `MENTIONS` links give a looser kind of information: which topics appear in the same articles. Appearing together is not a fact about diabetes, but it shows how Vinmec writes about the disease. Cardiovascular disease and hypoglycaemia come first simply because each is mentioned in a very large share of all articles. This is a known weakness of raw co-occurrence counts: pairs of common topics always rank high, whether or not they are related. Finding pairs that are *unusually* often together would require comparing each count with what is expected from how common each topic is on its own.

## 7.2 Correctness of the Facts

### 7.2.1 What was measured and how

The measure is **precision**: of the facts the graph contains, what share are correct. Precision was chosen because it is what a user of the graph experiences—every wrong fact is something the system will state with confidence. The procedure (`kg/evaluate.py`):

1. draw 50 of the extracted facts at random, with a fixed random seed (42) so that the same sample can be drawn again;
2. for each fact, show the article title and an evidence sentence from a section that the extractor actually used;

3. judge the fact by reading **all** its supporting articles, not only the sentence shown, and record `correct` and an `error_type` in `evaluation/manual_audit.csv`.

A fact counts as correct when the Vinmec articles state it for that type of diabetes. This is a check against the source, not against clinical guidelines; a medical review would be a different and much more expensive evaluation. Why random facts and why fifty? Random, because choosing the facts by hand would unconsciously favour easy ones and make the result look better than it is. Fifty, because each fact requires reading several articles; fifty is enough to estimate precision with useful accuracy and to reveal the main kinds of error.

Table 23: The first 12 of the 50 sampled facts with their judgement. Support is the value at the time of the audit, before the stricter heading rule of Section 9.

| # | Subject | Relation | Object | Support | Correct |
|---|---|---|---|---|---|
| 1 | Tiền đái tháo đường | HAS_COMPLICATION | Bệnh thần kinh ngoại biên | 5 | yes |
| 2 | Đái tháo đường thai kỳ | AVOIDS_FOOD | Thịt chế biến sẵn | 7 | yes |
| 3 | Đái tháo đường thai kỳ | HAS_RISK_FACTOR | Chế độ ăn nhiều đường, tinh bột | 2 | yes |
| 4 | Đái tháo đường | HAS_RISK_FACTOR | Rối loạn mỡ máu | 16 | **no** |
| 5 | Tiền đái tháo đường | HAS_RISK_FACTOR | Tuổi cao | 2 | yes |
| 6 | Đái tháo đường | HAS_COMPLICATION | Tăng nhãn áp | 9 | yes |
| 7 | Đái tháo đường type 2 | DIAGNOSED_BY | Đường huyết ngẫu nhiên | 4 | yes |
| 8 | Đái tháo đường | TREATED_BY | Saxagliptin | 5 | yes |
| 9 | Đái tháo đường type 1 | DIAGNOSED_BY | Xét nghiệm nước tiểu | 2 | yes |
| 10 | Đái tháo đường type 2 | TREATED_BY | Glimepiride | 2 | yes |
| 11 | Đái tháo đường | MANAGED_BY | Ngủ đủ giấc | 7 | yes |
| 12 | Đái tháo đường thai kỳ | AVOIDS_FOOD | Trái cây nhiều đường | 4 | yes |

### 7.2.2 Result

Table 24: Precision on the 50 sampled facts, by relation type.

| Relation | Sampled | Correct |
|---|---|---|
| AVOIDS_FOOD | 7 | 7 |
| DIAGNOSED_BY | 6 | 6 |
| HAS_COMPLICATION | 7 | 7 |
| HAS_RISK_FACTOR | 11 | 10 |
| HAS_SYMPTOM | 7 | 6 |
| MANAGED_BY | 2 | 2 |
| RECOMMENDS_FOOD | 3 | 3 |
| TREATED_BY | 7 | 7 |
| **All** | 50 | **48** (0.96; likely range 0.87–0.99) |

**48 of the 50 facts are correct, a precision of 0.96.** Both errors are in symptoms and risk factors, the two relation types that depend most on the section heading; the facts about complications, tests, drugs and foods were all correct.

How precise is "0.96"? With 50 facts, the true precision of the whole graph could be somewhat higher or lower: another random sample of 50 might have contained one error or four. The standard calculation for a proportion measured on a small sample (the Wilson interval) gives a likely range of **0.87 to 0.99** for the precision of the whole graph. In plain terms: roughly nine facts out of ten or better are correct, and "96%" should not be read as an exact figure. The two errors and what was done about them are analysed in Section 9.

### 7.3 How Much Evidence the Facts Have

Table 25: Number of extracted facts that remain when a minimum number of supporting articles is required.

| Minimum support | Facts | Share |
|---|---|---|
| $\geq 1$ | 301 | 100% |
| $\geq 2$ | 295 | 98% |
| $\geq 3$ | 252 | 84% |
| $\geq 5$ | 161 | 53% |
| $\geq 10$ | 70 | 23% |
| $\geq 20$ | 32 | 11% |

Table 25 shows how the graph shrinks as more evidence is required. The six facts with support 1 come from Vinmec's disease pages (Section 4.3). Requiring three articles keeps 84% of the facts; requiring ten keeps 23%, the core facts that Vinmec repeats across many articles. Because support is stored on every fact, this choice does not have to be made once for everybody: a query, or the user of the website, can choose the level of evidence appropriate for the purpose.

## 8 Demo

This section walks through the website as a user would, with screenshots of the running system. Each scenario corresponds to one of the questions of Table 2. All screenshots were taken from the live website connected to the Neo4j database described above.

### 8.1 Scenario 1: Finding an Entity

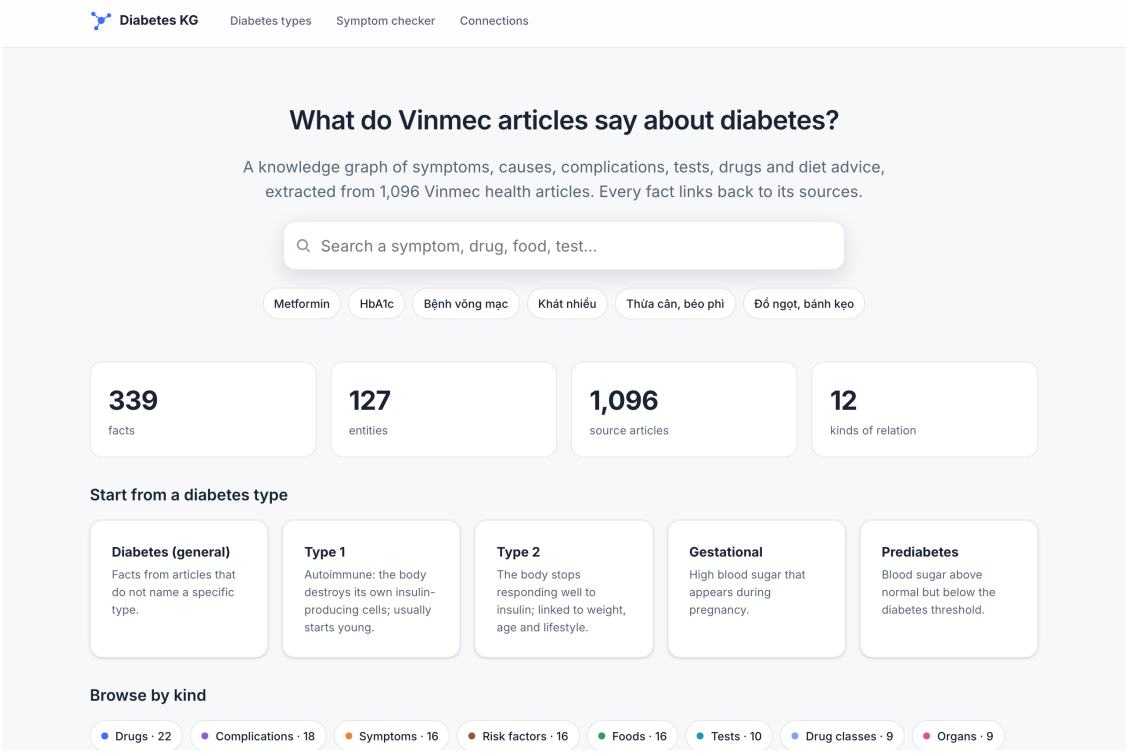



Figure 16: Home page: a search box, quick links to common entities, the size of the graph, and an entry point for each diabetes type.

The home page (Figure 16) tells the visitor in one sentence what the site is—what Vinmec articles say about diabetes, as a graph built from 1,096 articles—and offers three ways in: searching for an entity, clicking one

of the example entities, or starting from a type of diabetes. The four tiles summarise the graph: 339 facts, 127 entities, 1,096 source articles and 12 kinds of relation. A collapsible section explains in four steps how the graph was built and states its measured precision and that it is not medical advice.

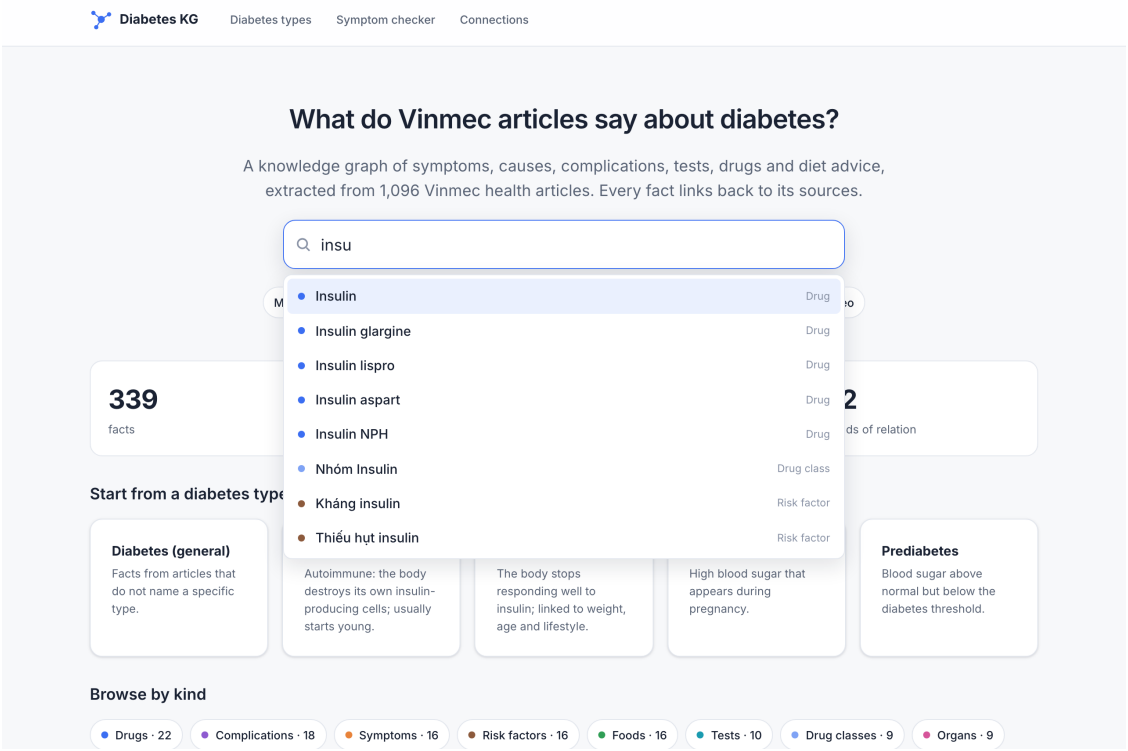



Figure 17: Searching “insu”: suggestions appear while typing, each with its type.

Typing “insu” (Figure 17) immediately lists every entity whose name contains these letters: the five insulin drugs, the insulin drug class, and two risk factors (insulin resistance and insulin deficiency). Each suggestion shows its type with a coloured dot, so *Kháng insulin* is not mistaken for a drug. Names that *start* with the typed letters come first, then the most connected entities. The list can be used with the keyboard (arrow keys and Enter), as in any standard search field. Each keystroke cancels the previous request, so fast typing never shows outdated results.

## 8.2 Scenario 2: Everything About One Entity, With Its Sources

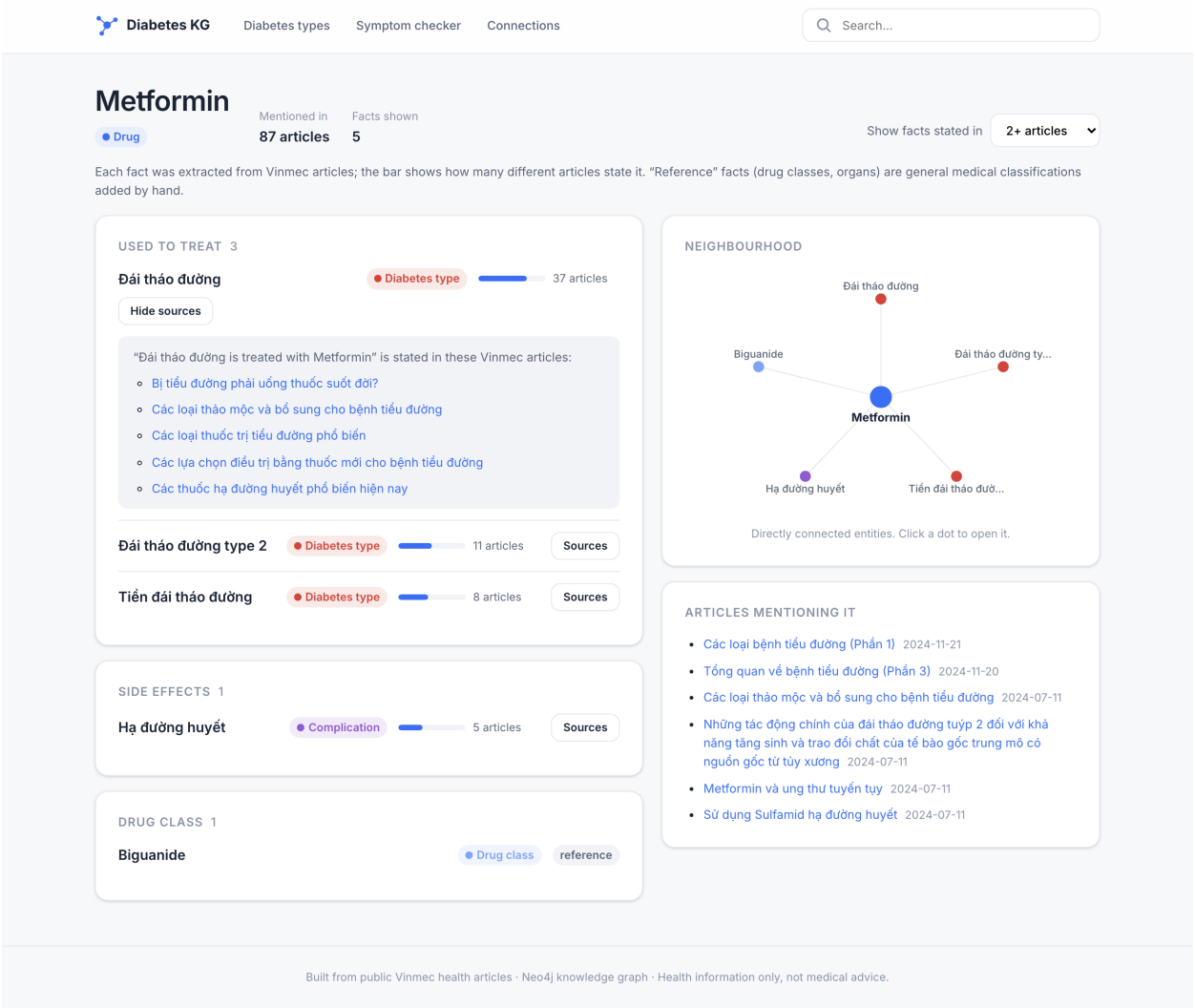



Figure 18: The Metformin page: facts grouped by relation, each with its number of supporting articles; the sources of the first fact are open. On the right, the neighbourhood drawing and the latest articles mentioning Metformin.

Selecting *Metformin* opens its page (Figure 18). The header shows its type, the number of articles that mention it (87) and the number of facts shown. The facts are grouped under plain-language headings derived from the relation and its direction: *Used to treat* (diabetes, type 2, prediabetes), *Side effects* (hypoglycaemia), *Drug class* (biguanide). Each fact has a bar and a number giving how many different articles state it; the bar uses a logarithmic scale so that a fact with 5 articles is still visible next to one with 37, but the number is always shown because the bar is only decoration.

Clicking *Sources* opens the evidence for one fact: the sentence "Đái tháo đường is treated with Metformin is stated in these Vinmec articles" followed by links to the five example articles. This is the provenance query of Section 7.1 made available to any reader. The drug class is labelled "reference" instead of having a bar, because it was added by hand.

The drawing on the right is the entity's one-hop neighbourhood (up to its twelve strongest neighbours), coloured by type; every dot is a link to that entity's page, so the graph can be explored by clicking from entity to entity. Below it, the latest articles that mention Metformin are listed with their dates.

The selector *Show facts stated in* (default: 2+ articles) lets the reader decide how much evidence a fact needs before it is shown, which is the precision/recall trade-off of Section 9.3 handed to the user.

## 8.3 Scenario 3: Comparing the Types of Diabetes

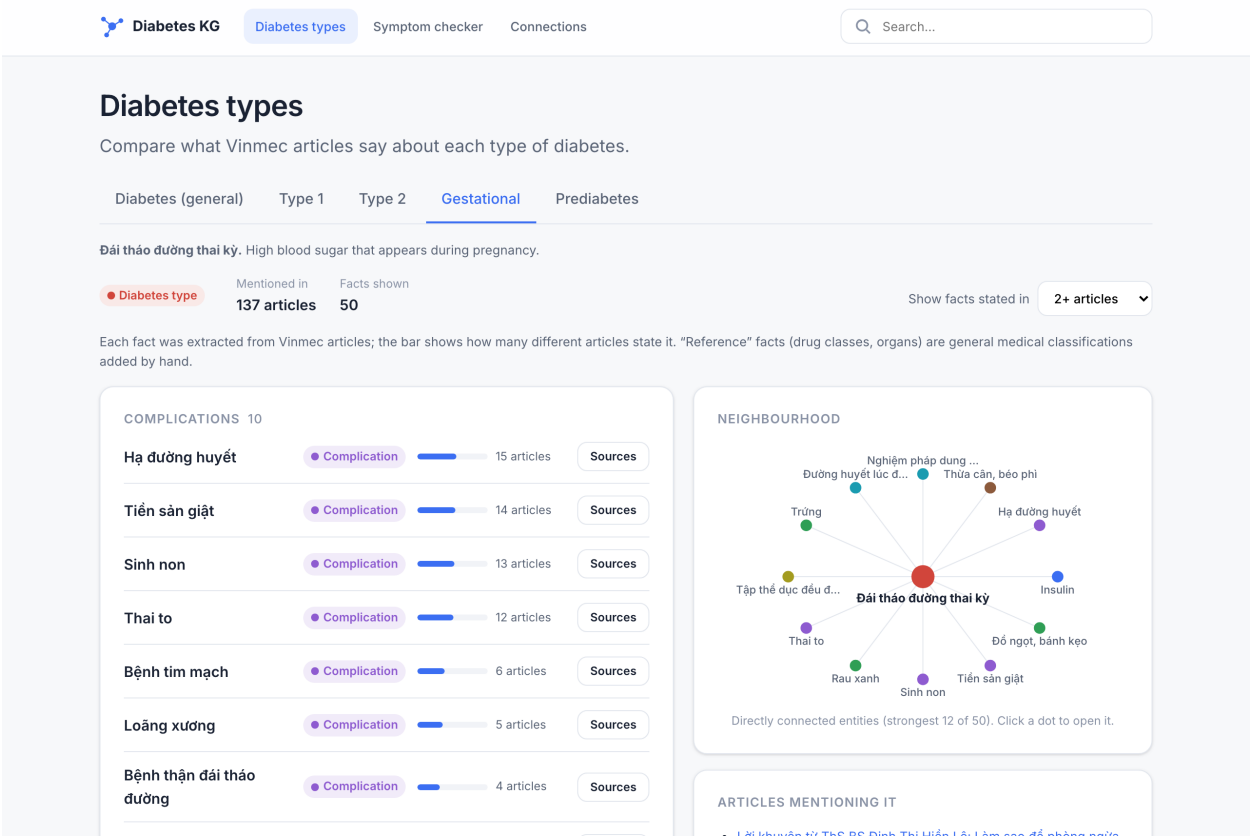



Figure 19: The *Diabetes types* page with the gestational tab open.

The *Diabetes types* page (Figure 19) puts the five types in tabs, each with a one-line plain explanation ("High blood sugar that appears during pregnancy"). Switching tabs shows how the facts differ. For gestational diabetes the complications list starts with hypoglycaemia (15 articles), pre-eclampsia (14), premature birth (13) and a large baby (12)—the pregnancy-specific pattern seen in the heatmap of Figure 13—and the neighbourhood drawing shows its foods, tests and risk factors at a glance.

## 8.4 Scenario 4: Checking Symptoms

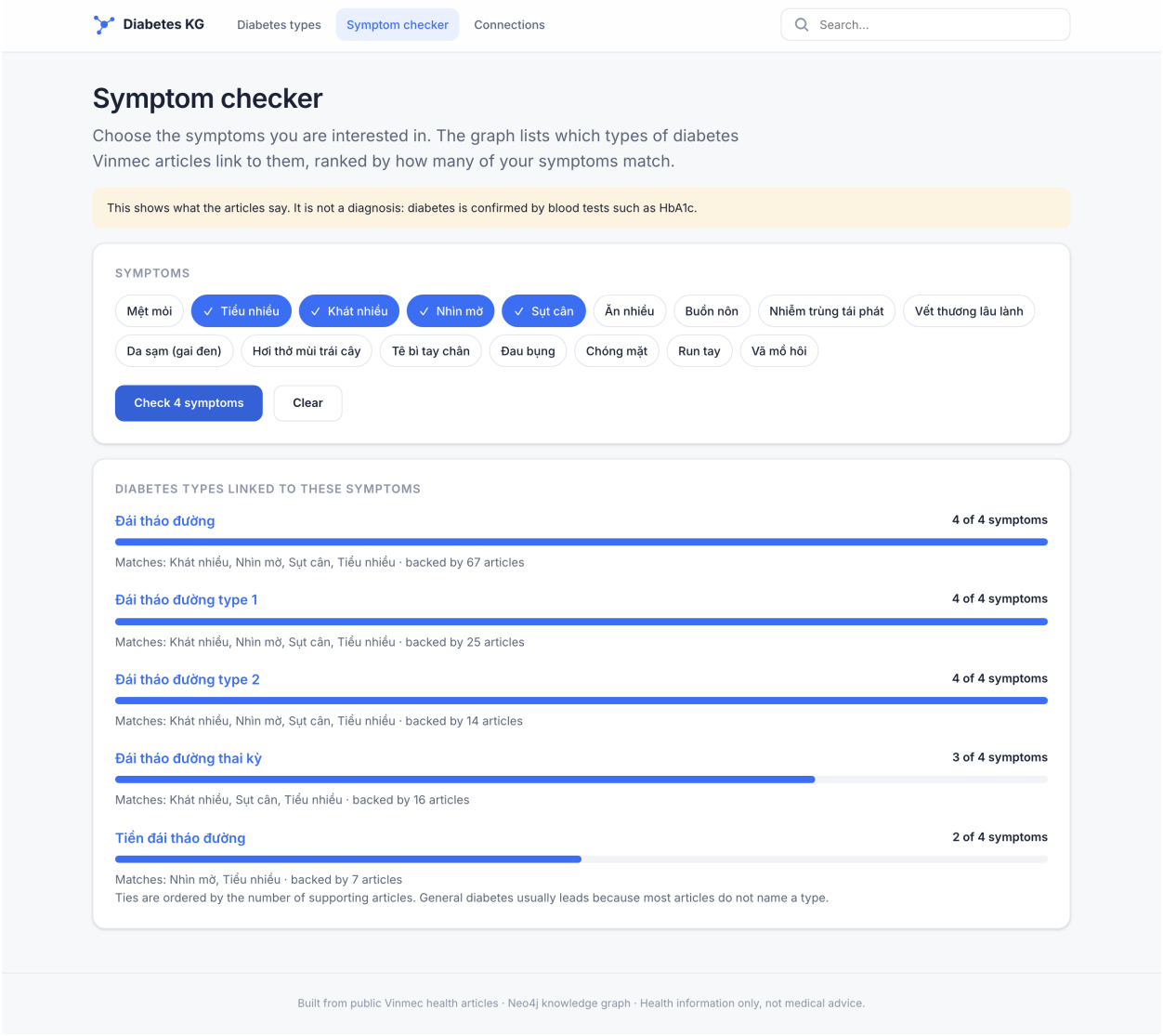



Figure 20: The symptom checker with four symptoms selected and the ranked result.

On the symptom checker (Figure 20) the user selects symptoms from the list of all 16 symptom entities and presses *Check*. The result ranks the diabetes types by how many of the selected symptoms the articles link to them ("4 of 4 symptoms"), with a bar for the share of matched symptoms, the list of matches and the number of supporting articles. A notice above the list states that this shows what the articles say and is not a diagnosis, because diabetes is confirmed by blood tests such as HbA1c. The result is the same as the Cypher query of Table 18, presented so that it can be understood without any knowledge of the graph.

## 8.5 Scenario 5: How Are Two Things Connected?

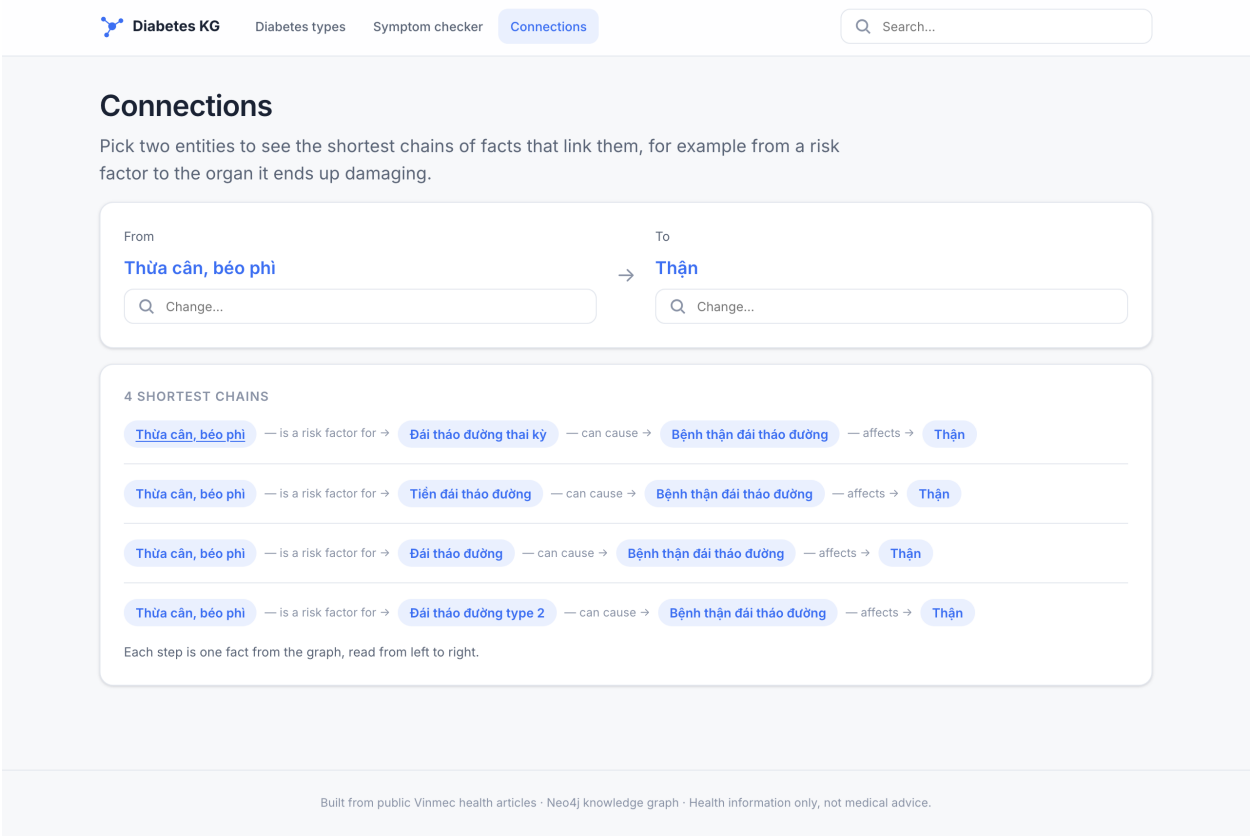



Figure 21: The *Connections* page: the shortest chains of facts from obesity (*Thừa cân, béo phì*) to the kidney (*Thận*).

The *Connections* page (Figure 21) asks for two entities and shows the shortest chains of facts between them. From obesity to the kidney there are four chains of three steps, one through each diabetes type that lists obesity as a risk factor, for example:

> *Thừa cân, béo phì* — is a risk factor for → *Đái tháo đường type 2* — can cause → *Bệnh thận đái tháo đường* — affects → *Thận*.

Each step is one fact, phrased in the right direction: the stored fact is "type 2 HAS_RISK_FACTOR obesity", and because the path walks it backwards the website writes "is a risk factor for". A chain like this is the kind of explanation a graph offers and a list of documents does not: each step comes from a different set of articles, and the graph connects them because they share a node.

### 8.6 On a Phone

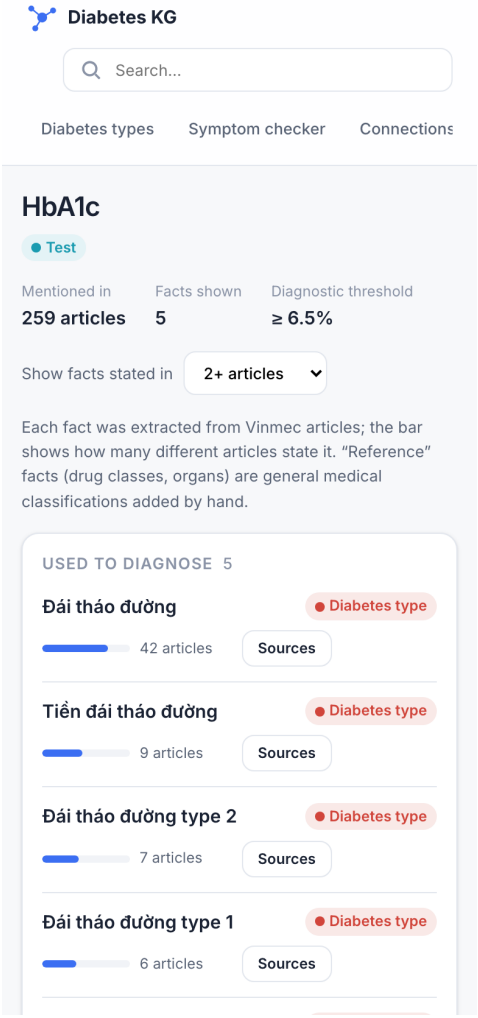



Figure 22: The HbA1c page on a phone-sized screen: the layout becomes a single column, the menu scrolls sideways.

The layout adapts to small screens (Figure 22): the two-column entity page becomes one column, the search box moves under the logo and the menu can be scrolled sideways. The HbA1c page also shows the diagnostic threshold ($\geq$ 6.5%) in the header, the one place where a node property is displayed.

## 9 Error Analysis

The quality check of Section 7.2 found two wrong facts out of fifty. This section explains why they were wrong, what was changed, what the change cost, and which other errors are known to remain.

## 9.1 The Two Errors in the Sample

Table 26: The two wrong facts in the sample and what the articles actually said.

| Wrong fact | What the supporting articles said |
|---|---|
| Đái tháo đường HAS_RISK_FACTOR Rối loạn mỡ máu (support 16) | High cholesterol is listed as a risk factor of the *kidney* and *eye* complications, under headings such as "Những ai dễ bị bệnh thận đái tháo đường?" and "Ai có nguy cơ mắc bệnh võng mạc đái tháo đường?", and as a cause of itching ("Một số nguyên nhân gây ngứa bàn tay và bàn chân") |
| Đái tháo đường HAS_SYMPTOM Đau bụng (support 18) | Abdominal pain is described as a symptom of *ketoacidosis* and of nerve damage in the gut, under headings such as "Biến chứng thần kinh" and "Kiểm soát biến chứng và phục hồi chức năng" |

Both errors have the same cause: a fact that belongs to a **complication** was attached to the disease itself. The section rule allowed it because the headings contained an allowed keyword (*nguy cơ*, *nguyên nhân*) or because complication sections were, in the first version, also allowed to produce symptoms. The rule looked at *what kind* of information a heading announces, but not *whose* symptoms or risk factors they are.

Two observations made the diagnosis clear. First, both wrong facts had high support (16 and 18), so the support filter could not catch them: many articles discuss the complications of diabetes, and each one contributed a wrong vote. A filter based on agreement only removes errors that are random; it cannot remove an error that the rules make systematically. Second, both errors were in the two relation types (symptoms and risk factors) where the owner of the information matters—a complication always belongs to diabetes, but a symptom may belong to diabetes or to one of its complications.

## 9.2 The Fix

The fix is the `about_disease()` check of Section 4.3.4: for symptoms and risk factors, the heading must name diabetes or be short and generic, must not name another topic (a complication, an organ, itching, stroke, coma, …), and must not be about reducing a risk; and complication sections no longer produce symptoms. Re-running the extraction changed the graph from 345 to 339 facts.

## 9.3 What the Fix Cost: Precision Versus Recall

Table 27: The six facts removed by the stricter rule.

| Removed fact | Was the removal right? |
|---|---|
| type 1 HAS_SYMPTOM Tê bì tay chân | yes: numbness is the nerve-damage complication, not a symptom of type 1 |
| type 2 HAS_SYMPTOM Vã mồ hôi | yes: sweating is a symptom of low blood sugar |
| Tiền đái tháo đường HAS_RISK_FACTOR Tuổi cao | no: a true fact (sample #5) was lost |
| Tiền đái tháo đường HAS_RISK_FACTOR Tăng huyết áp | no: a true fact (sample #15) was lost |
| Đái tháo đường HAS_RISK_FACTOR Dùng corticoid kéo dài | no: a true fact was lost |
| type 2 HAS_RISK_FACTOR Hút thuốc lá | no: a true fact was lost |

Two of the six removals were real errors that the sample had not happened to contain. The other four were true facts. Each had exactly one good supporting article and one weak one (for example, the age and blood-pressure facts for prediabetes came partly from the article "Chăm sóc các vấn đề tim mạch ở người bệnh

tiểu đường và tiền tiểu đường", whose risk-factor bullets sit under a heading about heart risk). The stricter rule removed the weak vote, the support fell to 1, and the minimum of two articles removed the fact. One consequence is visible in Table 6: the risk factor *Dùng corticoid kéo dài* lost its only fact and no longer appears in the graph.

And the two original errors? They stayed in the graph, but with far less support: 16 → 3 articles for high cholesterol and 18 → 6 for abdominal pain. Reading the remaining articles showed that they *do* state these facts under genuine diabetes headings: "Nguyên nhân và yếu tố nguy cơ gây bệnh tiểu đường: … Rối loạn mỡ máu", and abdominal pain in symptom lists for diabetes in children (where it is the sign of ketoacidosis at onset). By the definition used here—the articles state it—they are now properly supported. The problem had been the weak supporting articles, not the facts.

This is the **precision/recall trade-off** that every extraction system faces. Strict rules produce fewer wrong facts but miss more true ones; loose rules find more true facts but let in more wrong ones. No setting is best for every use. This is why the support count is kept on every fact instead of being used only once as a filter: a careful application can require three or five articles, while someone exploring the data can see everything. The website exposes exactly this choice to its users.

## 9.4 Other Known Errors

The sample of fifty facts cannot find every error. Inspecting the queries and the graph revealed four more.

**A wrong side effect.** The graph contains *Insulin HAS_SIDE_EFFECT Nhiễm toan ceton* (ketoacidosis), supported by five articles. It is wrong: ketoacidosis is caused by a *lack* of insulin and is *treated* with insulin. The filter of Section 4.3 skips sentences with *thiếu* or *không đủ*, but the articles connect insulin and ketoacidosis in other ways that the rule "drug + complication + *gây* in one sentence" cannot tell apart:

- insulin as the **treatment**: "nhiễm toan ceton … có thể *gây* nguy hiểm đến tính mạng nếu như không được xử trí kịp thời bằng insulin";
- another drug causing it **together with** insulin: "Danazol: dùng cùng với Insulin do có thể *gây* tăng glucose máu và nhiễm toan ceton";
- hormones that **work against** insulin: "Những hormone này … có tác dụng ngược lại với insulin … sẽ *gây* ra toan ceton".

In each sentence the word *gây* belongs to something other than insulin. Deciding *what* causes *what* requires understanding the grammar of the sentence, which cue words cannot do. Sentence-level relation extraction is the weakest part of the method; Section 11 proposes a language model for it.

**An alias with a second meaning.** The fruit alias *táo* (apple) also matches *táo bón* (constipation). Word boundaries cannot prevent this because *táo* is a whole word in both. The section rule limits the damage—a food only counts in diet sections—but a diet section that mentions constipation would produce a false vote for "low-sugar fruit".

**A concept hidden by a longer one.** The vocabulary contains *Hạ đường huyết sơ sinh* (hypoglycaemia of the newborn) separately from *Hạ đường huyết* (hypoglycaemia). Articles rarely use the full phrase; they say "trẻ sinh ra có thể bị hạ đường huyết", which matches the general entity. The newborn-specific entity therefore never reached two articles and is absent from the graph, while its facts are counted under the general hypoglycaemia (which is why hypoglycaemia is the top complication of gestational diabetes).

**Dominance of the general node.** Because the subject comes from the title only, the general diabetes node collects most facts (Figure 7). This is not an error in a single fact, but it makes type-specific questions less complete than they could be.

# 10 Limitations

The lecture lists four ways in which a Knowledge Graph can fail: missing entities, incorrect relationships, duplicate entities and outdated information. Each applies to this project in a specific way, and a few further limitations come from the design choices.

**Missing entities (unknown recall).** The vocabulary is fixed: 106 concepts written in 320 ways. Anything not in it can never be found, however often the articles discuss it—a newer drug or insulin brand, a rarer complication, a traditional remedy, a specific test. The evaluation measured how many extracted facts are correct (precision) but not how many true facts in the articles were missed (recall). Measuring recall would require annotating a set of articles by hand with every fact they contain, which was outside the scope of this project.

**Incorrect relationships.** About 4% of the sampled facts were wrong (likely range 1–13%), mostly facts about a complication attached to the disease. Section headings are a strong signal of meaning but not a perfect one, and the side-effect extraction from cue words in single sentences is weaker still (Section 9).

**Duplicates and granularity.** Aliases deliberately merge near-synonyms—*tê bì*, *dị cảm* and *kiến bò* all become *Tê bì tay chân*—and sometimes merge things a doctor would keep apart, such as hypoglycaemia of the mother and of the newborn. Conversely, a concept written in a way the vocabulary does not know is simply missed rather than duplicated, so true duplicates are rare.

**Outdated information.** All articles carry a 2024 publication date that reflects Vinmec's re-publication of its archive, not when each article was written, and an old article counts the same as a new one. Medical advice changes (new drug classes, revised targets), and the graph has no notion of time. A real system would re-crawl regularly and record the date of each fact.

**Coarse subjects.** The type of diabetes is taken only from the article title, so many facts that are really about type 2 are attached to the general diabetes node.

**Fixed food advice.** Whether a food is recommended or to be avoided is decided by the food, not read from the article, so conditional advice ("in moderate amounts", "choose low-sugar varieties") cannot be represented.

**One source.** All facts come from one hospital's website. This makes the graph consistent, but it inherits that source's emphases and gaps, and "correct" in this report means "stated by Vinmec", not "checked against current clinical guidelines".

**Hand-added facts.** Drug classes, affected organs and diagnostic thresholds were added by hand. They are clearly labelled `curated` and are standard medical knowledge, but they were not learned from the articles.

**Not medical advice.** The graph summarises public health articles. The symptom checker shows which types of diabetes the articles associate with a set of symptoms; it does not diagnose anyone. The website states this where it matters.

# 11 Conclusion and Future Work

## 11.1 What Was Achieved

This project built a Knowledge Graph of diabetes knowledge from Vietnamese health articles, stored it in Neo4j and made it explorable through a website.

- **Data (R1).** 1,197 candidate pages were found through Vinmec's sitemap; a relevance filter that works on the accented text removed accent collisions and off-topic pages, leaving 1,096 diabetes articles (1.13 million words), saved section by section.
- **Knowledge Graph (R2, R3).** An extractor that combines a hand-written vocabulary, the section headings of the articles and agreement between articles produced 127 entities of 10 types and 339 facts of 12 relation types, stored as `subject, relation, object` CSV with support and sources—well above the lecture's minimums of 50 entities, 100 relationships, 5 relation types and 3 entity types.
- **Neo4j and queries (R4, R6).** The graph is stored in Neo4j with its evidence on every relationship and its source articles as nodes. Fifteen Cypher queries, including one-hop, two-hop, aggregation, path and provenance queries, answer the questions the system was designed for; `graph_facts` retrieves the facts around an entity within one or two hops, and the evaluation showed why this bound is necessary (5 facts at one hop, 230 at two).
- **Visualisation (R5).** The graph was shown as a whole, by neighbourhood, by drug family and by diabetes type, and analysed with degree, directed and undirected PageRank, and article mentions.
- **Website (R7).** A small API and a React client let anyone search the graph, read the facts about an entity with their sources, compare diabetes types, check symptoms and find the chains of facts that connect two entities.
- **Evaluation (R8, R9).** 48 of 50 randomly sampled facts were correct (0.96; likely range 0.87–0.99). The two errors shared one cause, which was fixed; the fix's cost in lost true facts was measured and explained as a precision/recall trade-off.

The main lesson of the project is that **structure already present in the text can do most of the work**. Health articles are organised under standard headings, and using those headings to decide what kind of fact a sentence can contain, together with requiring independent articles to agree, produced a reliable graph without any trained model. The quality check also showed the limit of agreement: it removes random noise, but not an error the rules make systematically—only looking at the errors revealed that.

### 11.2 Future Work

1. **Measure recall.** Annotate a sample of articles by hand with every fact they state, and measure how many the extractor finds. This would quantify the main unknown of the project.
2. **Grow the vocabulary semi-automatically.** Frequent noun phrases that appear under symptom or treatment headings but match no vocabulary entry are candidates for new entities; a person would only need to approve them.
3. **Better relation extraction.** Use a language model to extract relations sentence by sentence ("does this sentence say that X causes Y, and is X taken or lacking?"), checked against the section rule. This would address the side-effect errors and allow new relations such as *food prevents complication*.
4. **Knowledge Graph + RAG.** Connect the graph to the retrieval-augmented chat system of Assignment 08: the graph supplies the facts and their paths, the article chunks supply detailed explanations, and the answer cites both.
5. **Freshness.** Re-crawl on a schedule, store the date of each article on its facts, and prefer recent evidence when articles disagree.

## A Reproduction

All commands are run from the `graph_01/` folder.

```
# environment
```

```
python -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt

# 1. crawl (about 20 minutes, resumable)  -> dataset/raw/pages/*.json,
↪  dataset/raw/index.csv
python kg/crawl.py

# 2. extract                                ->
↪  dataset/processed/{entities,triples,articles,mentions}.csv
python kg/extract.py

# 3. Neo4j (Docker) and load
docker run -d --name neo4j-diabetes -p 7474:7474 -p 7687:7687 \
    -e NEO4J_AUTH=neo4j/diabetes123 -v "$PWD/neo4j_data:/data" neo4j:5
python kg/load.py

# 4. retrieval, drawings, quality sample
python kg/retriever.py "Metformin" --hops 1
python kg/visualize.py
python kg/evaluate.py          # draws a new sample; overwrites
↪  evaluation/manual_audit.csv

# 5. notebook (kernel "Python (graph_01)")
jupyter nbconvert --to notebook --execute --inplace
↪  notebook/01_diabetes_kg.ipynb

# 6. website
uvicorn api.main:app --reload --port 8000      # API, docs at
↪  http://localhost:8000/docs
cd web && npm install && npm run dev           # http://localhost:5173

# 7. report
cd report && xelatex Graph_01.tex && xelatex Graph_01.tex
```

The Cypher queries of Section 7.1 are in `kg/queries.cypher` and can be pasted into Neo4j Browser at `http://localhost:7474` (user `neo4j`, password `diabetes123`).